cell-molecular-biology: the Cell and Molecular Biology program — user guide

cell-molecular-biology-ai999x 1.1.0, Rust build: command line, desktop app and browser. **Creator / engineer:** Ibnul Jaif Farabi · **Company:** AI999X · support@ai999x.com

Cell and Molecular Biology covers membranes and diffusion, osmosis and membrane potentials, cell growth and the cell cycle, DNA and RNA melting temperatures and quantification, sequences, PCR and qPCR, gene expression, enzyme kinetics, inhibition and binding, buffers and dilutions. This guide is about `cell-molecular-biology`, the model as a self-contained program for Windows and Linux: 94 capabilities in 7 topics, answered on your own computer with no network, no Python and nothing else to install. The same compiled model runs in the AI999X desktop app and in the browser on the model's page (https://www.ai999x.com/models/cell-molecular-biology-ai999x/); the Python package (`pip install cell-molecular-biology-ai999x`) has its own guide. Every command output printed here is what the program printed.

Contents

- 1. Get it: desktop app, command line, browser, Python
- 2. Quick start
- 3. Writing inputs
- 4. Outcomes and exit codes
- 5. Scripts and agents: batch and JSON
- 6. Figures
- 7. In a web page or Node.js: WebAssembly
- 8. Every capability

1. Get it

Version 1.4.0 of the AI999X desktop app and of the command-line bundle. Checksums for every file: https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS; the newest version is always under https://www.ai999x.com/downloads/desktop/latest/. Full instructions: https://www.ai999x.com/desktop-cli/

what	system	download
Desktop app: Cell and Molecular Biology	Windows x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-cell-molecular-biology-1.4.0-windows-x86_64.zip
Desktop app: Cell and Molecular Biology	Linux x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-cell-molecular-biology-1.4.0-linux-x86_64.tar.gz
Desktop app: all models	Windows x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-1.4.0-windows-x86_64.zip
Desktop app: all models	Linux x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
Command line	Windows x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-cli-1.4.0-windows-x86_64.zip
Command line	Linux x86-64	https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz

Desktop app

Cell and Molecular Biology in its own app: search, questions in words, input forms, live figures and explanations. Windows: Download ai999x-cell-molecular-biology-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x-cell-molecular-biology.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-cell-molecular-biology-1.4.0-linux-x86_64.tar.gz
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-cell-molecular-biology-1.4.0-linux-x86_64.tar.gz
./ai999x-cell-molecular-biology-1.4.0-linux-x86_64/ai999x-cell-molecular-biology
```

It opens on this model: type a question or pick a capability, fill in the form and press Compute. The AI999X desktop app with every model works the same way (choose the model first). Windows: Download ai999x-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-1.4.0-linux-x86_64.tar.gz
./ai999x-1.4.0-linux-x86_64/ai999x
```

Command line — Linux

The bundle holds one program per model; `cell-molecular-biology` is this one. The checksum line should print OK.

```
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz
curl -LO https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
mkdir -p ~/.local/share && tar xzf ai999x-cli-1.4.0-linux-x86_64.tar.gz -C ~/.local/share
echo 'export PATH="$HOME/.local/share/ai999x-cli-1.4.0-linux-x86_64:$PATH"' >> ~/.bashrc && source ~/.bashrc
cell-molecular-biology --version
```

Command line — Windows (PowerShell)

The fourth line should print True.

```
$F = "ai999x-cli-1.4.0-windows-x86_64.zip"
Invoke-WebRequest "https://www.ai999x.com/downloads/desktop/1.4.0/$F" -OutFile $F
Invoke-WebRequest "https://www.ai999x.com/downloads/desktop/1.4.0/SHA256SUMS" -OutFile SHA256SUMS
(Get-FileHash $F -Algorithm SHA256).Hash.ToLower() -eq ((Select-String -SimpleMatch $F SHA256SUMS).Line -split ' ')[0]
Expand-Archive $F -DestinationPath "$env:LOCALAPPDATA\AI999X" -Force
$D = "$env:LOCALAPPDATA\AI999X\ai999x-cli-1.4.0-windows-x86_64"; $env:Path += ";$D"
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path", "User") + ";$D", "User")
cell-molecular-biology --version
```

In your browser

Open https://www.ai999x.com/models/cell-molecular-biology-ai999x/, choose a capability and press Compute in your browser: the same program runs on your device as WebAssembly; nothing is sent anywhere.

Python

```
pip install cell-molecular-biology-ai999x
```

2. Quick start

2.1 Check the install

Prints the model, its version and that this is the Rust build.

```
$ cell-molecular-biology --version
cell-molecular-biology-ai999x 1.0.0 (Rust build, ax-engine 0.1.0)
```

2.2 Find a capability

Search the capabilities by what you want to know; the first column is the id to pass to info and solve.

```
$ cell-molecular-biology search "nernst equilibrium potential of an ion across a cell membrane"
cmb_nernst_potential                Nernst equilibrium potential of an ion across a cell membrane
(membranes and diffusion / membrane potential)
cmb_cell_equilibration              Equilibration of a solute across a spherical cell's membrane
(membranes and diffusion / permeability)
cmb_donnan_equilibrium              Donnan equilibrium with an impermeant anion  (membranes and
diffusion / membrane potential)
cmb_channel_current                 Macroscopic current through ion channels  (membranes and
diffusion / ion channels)
cmb_cell_volume_osmotic             Cell volume after an osmotic change (Boyle-van 't Hoff)
(membranes and diffusion / osmosis)
cmb_ionic_strength                  Ionic strength of a solution  (solution biochemistry / ionic
strength)
cmb_membrane_permeability           Membrane permeability coefficient and flux  (membranes and
diffusion / permeability)
cmb_osmolarity_tonicity             Osmolarity of a solution and its tonicity for a cell
(membranes and diffusion / osmosis)
… 12 more
```

2.3 List the capabilities

Every capability the program answers: its id (what you pass to solve) and its name.

```
$ cell-molecular-biology list
cmb_absorbance_ratios                    A260/A280 and A260/A230 purity ratios
cmb_binding_free_energy                  Standard binding free energy from Kd
cmb_buffer_after_addition                Buffer pH after adding strong acid or base
cmb_buffer_capacity                      Buffer capacity of a weak-acid buffer
cmb_buffer_preparation                   Recipe for a buffer of given pH and concentration
cmb_c1v1_dilution                        Dilution of a stock solution (C1 V1 = C2 V2)
cmb_cell_cycle_age_corrected             Cell-cycle phase durations corrected for the exponential
age distribution
cmb_cell_cycle_phase_durations           Cell-cycle phase durations from phase fractions
… 86 more
```

2.4 What a capability needs

The equation or method, every input and output with its SI unit. Read this before solve: it tells you the input names.

```
$ cell-molecular-biology info cmb_nernst_potential
Nernst equilibrium potential of an ion across a cell membrane
  E = (R T / z F) ln(C_out / C_in)
  in   z        charge number of the ion (e.g. 1 for K+, 2 for Ca2+, -1 for Cl-) (integer) []
  in   C_out    outside concentration [mol/m^3]
  in   C_in     inside concentration [mol/m^3]
  in   T        absolute temperature (degC is converted) [K]
  out  E        equilibrium potential (inside minus outside) [V]
  out  E_mV     equilibrium potential in mV []
```

2.5 Solve

Inputs as name=value, each with any unit of the right kind. The answer lists every output, the method used, each input converted to SI, and the conditions the answer assumes.

```
$ cell-molecular-biology solve cmb_nernst_potential z=1 "C_out=5 (mmol/L)" "C_in=140 (mmol/L)"
"T=37 degC"
E = -0.08905869404 V
E_mV = -89.05869404
  by: Nernst equilibrium potential of an ion across a cell membrane  (E = (R T / z F) ln(C_out /
C_in))
  C_out = 5 mol/m^3
  C_in = 140 mol/m^3
  T = 310.15 K
  assumes: z a non-zero whole number; concentrations > 0; T > 0
```

2.6 Machine-readable answer

--json prints the same answer as one line of JSON (values in SI): for scripts and other programs.

```
$ cell-molecular-biology solve cmb_nernst_potential z=1 "C_out=5 (mmol/L)" "C_in=140 (mmol/L)"
"T=37 degC" --json
{"outcome":"SOLVED","message":"","value":-0.0890586940367,"unit":"V","solved_for":"E","si":
[["C_out",5.0],["C_in",140.0],["T",310.15]],"values":[["E",-0.0890586940367,"V"],
["E_mV",-89.0586940367,""]],"target":"E","phrase":"equilibrium potential (inside minus
outside)","assumptions":["z a non-zero whole number","concentrations > 0","T > 0"],"name":"Nernst
equilibrium potential of an ion across a cell membrane","equation":"E = (R T / z F) ln(C_out /
C_in)"}
```

2.7 When an input is missing

No number is guessed: the answer names what is missing, shows a complete call, and the program exits with status 1.

```
$ cell-molecular-biology solve cmb_nernst_potential z=1 "C_out=5 (mmol/L)" "C_in=140 (mmol/L)"
NEEDS_INPUT: cmb_nernst_potential needs T (absolute temperature (degC is converted)). Example:
solve('cmb_nernst_potential', z=1, C_out='5 mol/m^3', C_in='140 mol/m^3', T='310.15 K').
```

2.8 Draw the figures

The live figures for your own inputs, written as SVG files into the folder given with --out (here the current folder); open them in any browser. Without --out the SVG text comes back as JSON.

```
$ cell-molecular-biology figures cmb_nernst_potential z=1 "C_out=5 (mmol/L)" "C_in=140 (mmol/L)"
"T=37 degC" --out .
[
  {
    "slug": "cmb-nernst-potential-e-vs-c-out",
    "title": "Nernst potential against the outside concentration",
    "caption": "E (equilibrium potential (inside minus outside)) against C_out (outside
concentration) from 0.5 to 50 (logarithmic axis across), the other inputs as given: 31 points, each
a run of the model. Marked: C_out = 5, E = -0.08906. Your inputs: z = 1, C_out = 5 (mmol/L), C_in =
140 (mmol/L), T = 37 degC.",
    "file": ".\\cmb-nernst-potential-e-vs-c-out.svg"
  },
  {
    "slug": "cmb-nernst-potential-live-1",
    "title": "Equilibrium potential (inside minus outside) as charge number of the ion (e.g. 1 for
K+, 2 for... varies",
    "caption": "Equilibrium potential (inside minus outside) as z runs from 1 to 3, every other
input as you entered it (3 solutions). Marked: your z = 1 gives E = -0.08906. Your inputs: z = 1,
C_out = 5 (mmol/L), C_in = 140 (mmol/L), T = 37 degC.",
    "file": ".\\cmb-nernst-potential-live-1.svg"
  },
  {
    "slug": "cmb-nernst-potential-live-2",
    "title": "Equilibrium potential (inside minus outside) as absolute temperature (degC is
converted) varies",
    "caption": "Equilibrium potential (inside minus outside) as T runs from 9.25 to 64.75 degC,
every other input as you entered it (29 solutions). Marked: your T = 37 degC gives E = -0.08906.
Your inputs: z = 1, C_out = 5 (mmol/L), C_in = 140 (mmol/L), T = 37 degC.",
    "file": ".\\cmb-nernst-potential-live-2.svg"
  }
]
```

2.9 Many questions at once

One JSON request per line in, one JSON answer per line out, in order: for scripts, notebooks and agents (the program starts once). In PowerShell: Get-Content cases.jsonl | cell-molecular-biology batch

`cases.jsonl`:

```
{"key": "cmb_nernst_potential", "inputs": {"z": "1", "C_out": "5 (mmol/L)", "C_in": "140 (mmol/L)",
"T": "37 degC"}}
{"key": "cmb_tm_nearest_neighbor", "inputs": {"seq": "CGTTCCAAAGATGTGGGCATGAGCTTAC", "Na": "50
(mmol/L)", "c1": "250 (nmol/L)", "c2": "250 (nmol/L)"}}
```

```
$ cell-molecular-biology batch < cases.jsonl
{"outcome":"SOLVED","message":"","value":-0.0890586940367,"unit":"V","solved_for":"E","si":
[["C_out",5.0],["C_in",140.0],["T",310.15]],"values":[["E",-0.0890586940367,"V"],
["E_mV",-89.0586940367,""]],"target":"E","phrase":"equilibrium potential (inside minus
outside)","assumptions":["z a non-zero whole number","concentrations > 0","T > 0"],"name":"Nernst
equilibrium potential of an ion across a cell membrane","equation":"E = (R T / z F) ln(C_out /
C_in)"}
{"outcome":"SOLVED","message":"","value":62.5816284138,"unit":"","solved_for":"Tm","si":
[["Na",50.0],["c1",0.00025],["c2",0.00025]],"values":[["Tm",62.5816284138,""],["dH",-221.7,""],
["dS",-628.76559587,""],["dG37",-26.6883504409,""]],"target":"Tm","phrase":"melting temperature in
degC","assumptions":["DNA letters, length >= 2","Na+ > 0","c1 > c2/2 (self-complementary: k =
c1)"],"name":"Nearest-neighbour melting temperature of a DNA duplex","equation":"sum nearest-
neighbour dH and dS with initiation and terminal A/T terms (+ symmetry); dS += 0.368 (N-1) ln[Na+];
Tm = 1000 dH / (dS + R ln k) - 273.15, k = c1 - c2/2"}
```

3. Writing inputs

Inputs are written `name=value`, one per argument; `cell-molecular-biology info <capability>` lists the names. Quote an argument that holds a space or brackets (bash, zsh and PowerShell all accept double quotes). The answer always states each input as it was read, converted to SI, so a misread unit is visible at once.

kind	example	how it is read
a number with a unit	`"R=0.128 nm"`	any unit of the same kind; converted to SI
a number in SI	`R=1.28e-10`	no unit means the SI unit shown by info
a choice	`structure=FCC`	one of the values info lists
a list or vector	`"v=[1, 2, 3]" or "v=1, 2, 3"`	brackets keep it a list
a matrix	`"A=[[1, 2], [3, 4]]"`	one list per row
true / false	`inclusive=true`	for yes/no options

An equation can be solved for any of its quantities: give the others and add `--target <symbol>`. Without `--target`, the program solves for the quantity the capability is named after.

4. Outcomes and exit codes

The program never guesses a number: when it cannot answer, it says why and exits with status 1, so a script can tell an answer from a refusal without reading the text.

outcome	meaning	exit status
SOLVED	the answer: every output with its unit, the method, the inputs in SI, the conditions assumed	0
NEEDS_INPUT	an input is missing; the message names it and shows a complete call	1
INVALID_INPUT	a value or unit of the wrong kind, an unknown input name, or a value outside the conditions	1
AMBIGUOUS	the inputs allow two different answers; say which one is meant	1
OUT_OF_SCOPE	the model has no capability for this, or the result would not be meaningful	1
(usage)	a command the program does not know: it prints its usage	2

5. Scripts and agents: batch and JSON

`cell-molecular-biology solve ... --json` prints one answer as one line of JSON. `cell-molecular-biology batch` reads one request per line on standard input and writes one JSON answer per line, in order, starting the program once for any number of requests. A request is `{"key": "<capability>", "inputs": {"name": "value", ...}, "target": null}`. In PowerShell, pipe the file in: `Get-Content cases.jsonl | cell-molecular-biology batch`. `cell-molecular-biology catalog` prints every capability with its inputs as JSON, to build forms or tool definitions.

answer field	meaning
outcome	one of the outcomes above
message	why there is no answer (empty when SOLVED)
value, unit	the main result in SI units
values	every output: [symbol, value, unit] (procedures)
target	the quantity solved for
si	each numeric input converted to SI: [symbol, value]
assumptions	the conditions the answer relies on
name, equation	the capability and its equation or method

```
import json, subprocess

reqs = [{"key": "<capability>", "inputs": {"name": "value"}}]
out = subprocess.run(["cell-molecular-biology", "batch"], input="\n".join(map(json.dumps, reqs)),
                     capture_output=True, text=True, check=True).stdout
answers = [json.loads(line) for line in out.splitlines()]
```

6. Figures

`cell-molecular-biology figures <capability> name=value ... --out DIR` draws the live figures for those inputs and writes each as an SVG file into DIR (an existing folder); without `--out` the SVG text comes back as JSON. The desktop app and the model page draw the same figures as soon as you press Compute.

```
$ cell-molecular-biology figures cmb_nernst_potential z=1 "C_out=5 (mmol/L)" "C_in=140 (mmol/L)"
"T=37 degC" --out .
[
  {
    "slug": "cmb-nernst-potential-e-vs-c-out",
    "title": "Nernst potential against the outside concentration",
    "caption": "E (equilibrium potential (inside minus outside)) against C_out (outside
concentration) from 0.5 to 50 (logarithmic axis across), the other inputs as given: 31 points, each
a run of the model. Marked: C_out = 5, E = -0.08906. Your inputs: z = 1, C_out = 5 (mmol/L), C_in =
140 (mmol/L), T = 37 degC.",
    "file": ".\\cmb-nernst-potential-e-vs-c-out.svg"
  },
  {
    "slug": "cmb-nernst-potential-live-1",
    "title": "Equilibrium potential (inside minus outside) as charge number of the ion (e.g. 1 for
K+, 2 for... varies",
    "caption": "Equilibrium potential (inside minus outside) as z runs from 1 to 3, every other
input as you entered it (3 solutions). Marked: your z = 1 gives E = -0.08906. Your inputs: z = 1,
C_out = 5 (mmol/L), C_in = 140 (mmol/L), T = 37 degC.",
    "file": ".\\cmb-nernst-potential-live-1.svg"
  },
  {
    "slug": "cmb-nernst-potential-live-2",
    "title": "Equilibrium potential (inside minus outside) as absolute temperature (degC is
converted) varies",
    "caption": "Equilibrium potential (inside minus outside) as T runs from 9.25 to 64.75 degC,
every other input as you entered it (29 solutions). Marked: your T = 37 degC gives E = -0.08906.
Your inputs: z = 1, C_out = 5 (mmol/L), C_in = 140 (mmol/L), T = 37 degC.",
    "file": ".\\cmb-nernst-potential-live-2.svg"
  }
]
```

7. In a web page or Node.js: WebAssembly

The same model is a WebAssembly module, `cell_molecular_biology_ai999x.wasm` (https://www.ai999x.com/compute/rust/cell_molecular_biology_ai999x.wasm), with a small loader, `ax-wasm.js` (https://www.ai999x.com/compute/rust/ax-wasm.js). It runs in a browser, a Web Worker or Node.js, with no server.

```
<script src="ax-wasm.js"></script>
<script>
  AXWasm.load("cell_molecular_biology_ai999x.wasm").then(function (m) {
    var a = m.solve({key: "<capability>", inputs: {name: "value"}});
    console.log(a.outcome, a.values || a.value);
    console.log(m.catalog().capabilities.length + " capabilities");
  });
</script>
```

8. Every capability

By topic. For each: what it does, what it needs and gives, when it applies, and a command with what the program printed for it. `cell-molecular-biology info <id>` shows the same inputs and outputs in the terminal.

Cell growth (14)

Batch culture time under Monod kinetics — `cmb_monod_batch_time`

The time for a batch culture (constant yield Y, Monod growth) to grow from biomass X0 to X, and the substrate left then.

$$t = \frac{1}{\mu_{max}}\left[\left(1+\frac{K_s Y}{B}\right)\ln\frac{X}{X_0} - \frac{K_s Y}{B}\ln\frac{B-X}{B-X_0}\right]$$

symbol		meaning	SI unit
`X0`	in	initial biomass concentration	kg/m^3
`S0`	in	initial substrate concentration	kg/m^3
`Y`	in	biomass yield (g biomass per g substrate)	—
`mu_max`	in	maximum specific growth rate	1/s
`Ks`	in	half-saturation constant	kg/m^3
`X`	in	target biomass concentration	kg/m^3
`t`	out	time to reach X	s
`t_h`	out	time in hours	—
`S`	out	substrate remaining	kg/m^3
`S_g_L`	out	substrate remaining in g/L	—

Applies when: all inputs > 0; X0 < X < X0 + Y S0; no maintenance or death.

```
$ cell-molecular-biology solve cmb_monod_batch_time "X0=0.1 g/L" "S0=10 g/L" Y=0.5 "mu_max=0.72
1/h" "Ks=0.2 g/L" "X=4 g/L"
t = 18954.49601 s
t_h = 5.265137782
S = 2.2 kg/m^3
S_g_L = 2.2
  by: Batch culture time under Monod kinetics  (S = S0 - (X - X0)/Y; integrate dX/dt = mu_max S
X/(Ks + S): t = [ln(X/X0) + (Ks Y/B)(ln(X/X0) - ln((B - X)/(B - X0)))]/mu_max, B = X0 + Y S0)
  X0 = 0.1 kg/m^3
  S0 = 10 kg/m^3
  Y = 0.5
  mu_max = 0.0002 1/s
  Ks = 0.2 kg/m^3
  X = 4 kg/m^3
  assumes: all inputs > 0; X0 < X < X0 + Y S0; no maintenance or death
```

In the worked example, X0 = 0.1 g/L, S0 = 10 g/L, Y = 0.5, mu_max = 0.72 1/h, Ks = 0.2 g/L, X = 4 g/L go through the equation above and give t = 18954.49601 s, t_h = 5.265137782, S = 2.2 kg/m^3, S_g_L = 2.2. It applies only when all inputs > 0; X0 < X < X0 + Y S0; no maintenance or death: outside that it says what is wrong instead of returning a number.

Cell concentration from a hemocytometer count — `cmb_hemocytometer_count`

The cell concentration from counts in the large (1 mm x 1 mm x 0.1 mm) squares of a hemocytometer, the dilution factor and, optionally, the total cells in a suspension volume.

$$c = \bar{n} \times d \times 10^{4}\ \mathrm{mL}^{-1}$$

symbol		meaning	SI unit
`counts`	in	cells counted in each large square	—
`dilution`	in	dilution factor of the counted sample (1 for undiluted, 2 for 1:1 trypan blue)	—
`V`	in	volume of the cell suspension (optional) (optional)	m^3
`mean_count`	out	mean count per large square	—
`cells_per_mL`	out	cells per mL	—
`total_cells`	out	total cells in V (when V is given)	—

Applies when: counts >= 0; dilution >= 1; large squares of 0.1 uL.

```
$ cell-molecular-biology solve cmb_hemocytometer_count "counts=[52, 48, 55, 45]" dilution=2
mean_count = 50
cells_per_mL = 1000000
  by: Cell concentration from a hemocytometer count  (cells/mL = mean count x dilution x 10^4)
  dilution = 2
  assumes: counts >= 0; dilution >= 1; large squares of 0.1 uL
```

In the worked example, counts = [52, 48, 55, 45], dilution = 2 go through the equation above and give mean_count = 50, cells_per_mL = 1000000. It applies only when counts >= 0; dilution >= 1; large squares of 0.1 uL: outside that it says what is wrong instead of returning a number.

Cell-cycle phase durations corrected for the exponential age distribution — `cmb_cell_cycle_age_corrected`

The durations of G1, S, G2 and M from phase fractions of an exponentially growing asynchronous population, using the age distribution n(a) = 2 ln2/T 2^(-a/T) (young cells are over-represented).

$$F(a) = 2\left(1 - 2^{-a/T}\right), \quad a_k = -T\log_2\left(1 - \frac{F_k}{2}\right)$$

symbol		meaning	SI unit
`Tc`	in	total cell-cycle (doubling) time	s
`fractions`	in	fractions of cells in G1, S, G2 and M (in this order, summing to 1)	—
`G1_h`	out	G1 duration in hours	—
`S_h`	out	S duration in hours	—
`G2_h`	out	G2 duration in hours	—
`M_h`	out	M duration in hours	—

Applies when: Tc > 0; four positive fractions summing to 1; balanced exponential growth.

```
$ cell-molecular-biology solve cmb_cell_cycle_age_corrected "Tc=24 h" "fractions=[0.45, 0.35, 0.15,
0.05]"
G1_h = 8.825562828
S_h = 8.861611432
G2_h = 4.623481871
M_h = 1.689343869
  by: Cell-cycle phase durations corrected for the exponential age distribution  (fraction younger
than age a: F(a) = 2(1 - 2^(-a/T)); phase end ages a_k = -T log2(1 - F_k/2) from cumulative
fractions)
  Tc = 86400 s
  assumes: Tc > 0; four positive fractions summing to 1; balanced exponential growth
```

In the worked example, Tc = 24 h, fractions = [0.45, 0.35, 0.15, 0.05] go through the equation above and give G1_h = 8.825562828, S_h = 8.861611432, G2_h = 4.623481871, M_h = 1.689343869. It applies only when Tc > 0; four positive fractions summing to 1; balanced exponential growth: outside that it says what is wrong instead of returning a number.

Cell-cycle phase durations from phase fractions — `cmb_cell_cycle_phase_durations`

The durations of G1, S, G2 and M from the fraction of an asynchronous population in each phase (simple proportion to the cycle time).

$$t_{phase} = f_{phase}\, T_c$$

symbol		meaning	SI unit
`Tc`	in	total cell-cycle time	s
`fractions`	in	fractions of cells in G1, S, G2 and M (in this order, summing to 1)	—
`G1_h`	out	G1 duration in hours	—
`S_h`	out	S duration in hours	—
`G2_h`	out	G2 duration in hours	—
`M_h`	out	M duration in hours	—

Applies when: Tc > 0; four positive fractions summing to 1.

```
$ cell-molecular-biology solve cmb_cell_cycle_phase_durations "Tc=24 h" "fractions=[0.45, 0.35,
0.15, 0.05]"
G1_h = 10.8
S_h = 8.4
G2_h = 3.6
M_h = 1.2
  by: Cell-cycle phase durations from phase fractions  (duration of a phase = fraction x Tc)
  Tc = 86400 s
  assumes: Tc > 0; four positive fractions summing to 1
```

In the worked example, Tc = 24 h, fractions = [0.45, 0.35, 0.15, 0.05] go through the equation above and give G1_h = 10.8, S_h = 8.4, G2_h = 3.6, M_h = 1.2. It applies only when Tc > 0; four positive fractions summing to 1: outside that it says what is wrong instead of returning a number.

Colony-forming units per mL from a plate count — `cmb_plate_count`

The viable count (CFU per mL) of the original culture from the colonies on a plate, the dilution factor of the plated sample and the plated volume.

$$\mathrm{CFU/mL} = \frac{n \times DF}{V_{plated}}$$

symbol		meaning	SI unit
`colonies`	in	colonies counted on the plate	—
`dilution`	in	dilution factor of the plated sample (1e6 for a 10^-6 dilution)	—
`V`	in	volume plated	m^3
`cfu_per_mL`	out	colony-forming units per mL of the original culture	—
`log10_cfu_per_mL`	out	log10 of CFU per mL (0 when no colonies)	—

Applies when: colonies a whole number; dilution >= 1; V > 0.

```
$ cell-molecular-biology solve cmb_plate_count colonies=150 dilution=1000000 "V=0.1 mL"
cfu_per_mL = 1500000000
log10_cfu_per_mL = 9.176091259
  by: Colony-forming units per mL from a plate count  (CFU/mL = colonies x dilution factor / volume
plated (mL))
  dilution = 1000000
  V = 1e-07 m^3
  assumes: colonies a whole number; dilution >= 1; V > 0
```

In the worked example, colonies = 150, dilution = 1000000, V = 0.1 mL go through the equation above and give cfu_per_mL = 1500000000, log10_cfu_per_mL = 9.176091259. It applies only when colonies a whole number; dilution >= 1; V > 0: outside that it says what is wrong instead of returning a number.

Doubling time and number of generations from two counts — `cmb_doubling_time_from_counts`

The doubling (generation) time, the number of generations and the specific growth rate from the cell counts at the start and end of an interval of exponential growth.

$$n = \log_2 \frac{N}{N_0}, \quad t_d = \frac{t}{n}, \quad \mu = \frac{\ln(N/N_0)}{t}$$

symbol		meaning	SI unit
`N0`	in	initial cell count (or concentration)	—
`N`	in	final cell count (same unit)	—
`t`	in	elapsed time	s
`td`	out	doubling time	s
`td_h`	out	doubling time in hours	—
`generations`	out	number of generations log2(N/N0)	—
`mu`	out	specific growth rate	1/s
`mu_per_h`	out	specific growth rate per hour	—

Applies when: N > N0 > 0; t > 0; exponential growth over the interval.

```
$ cell-molecular-biology solve cmb_doubling_time_from_counts N0=10000 N=160000 "t=8 h"
td = 7200 s
td_h = 2
generations = 4
mu = 9.627044174e-05 1/s
mu_per_h = 0.3465735903
  by: Doubling time and number of generations from two counts  (n = log2(N/N0) = 3.32 log10(N/N0);
td = t/n; mu = ln 2 / td)
  N0 = 10000
  N = 160000
  t = 28800 s
  assumes: N > N0 > 0; t > 0; exponential growth over the interval
```

In the worked example, N0 = 10000, N = 160000, t = 8 h go through the equation above and give td = 7200 s, td_h = 2, generations = 4, mu = 9.627044174e-05 1/s. It applies only when N > N0 > 0; t > 0; exponential growth over the interval: outside that it says what is wrong instead of returning a number.

Exponential growth of a cell population — `cmb_exponential_growth`

The number of cells after a time t of exponential growth from N0 with doubling time td, the number of generations and the specific growth rate.

$$N = N_0\,2^{t/t_d} = N_0 e^{\mu t}, \quad \mu = \frac{\ln 2}{t_d}$$

symbol		meaning	SI unit
`N0`	in	initial number (or concentration) of cells	—
`td`	in	doubling (generation) time	s
`t`	in	elapsed time	s
`N`	out	number of cells at time t	—
`generations`	out	number of generations t/td	—
`fold`	out	fold increase N/N0	—
`mu`	out	specific growth rate ln2/td	1/s
`mu_per_h`	out	specific growth rate per hour	—

Applies when: N0, td > 0; t >= 0; balanced exponential growth (no lag, no limitation).

```
$ cell-molecular-biology solve cmb_exponential_growth N0=1000 "td=0.3333333333 h" "t=2 h"
N = 64000.00003
generations = 6.000000001
fold = 64.00000003
mu = 0.0005776226505 1/s
mu_per_h = 2.079441542
  by: Exponential growth of a cell population  (N = N0 2^(t/td); mu = ln 2 / td)
  N0 = 1000
  td = 1200 s
  t = 7200 s
  assumes: N0, td > 0; t >= 0; balanced exponential growth (no lag, no limitation)
```

In the worked example, N0 = 1000, td = 0.3333333333 h, t = 2 h go through the equation above and give N = 64000.00003, generations = 6.000000001, fold = 64.00000003, mu = 0.0005776226505 1/s. It applies only when N0, td > 0; t >= 0; balanced exponential growth (no lag, no limitation): outside that it says what is wrong instead of returning a number.

Logistic growth of a population — `cmb_logistic_growth`

The population at time t under logistic growth with carrying capacity K and intrinsic rate r, the growth rate dN/dt there and the time of the inflection point (N = K/2).

$$N(t) = \frac{K}{1 + \frac{K - N_0}{N_0} e^{-rt}}, \qquad \frac{dN}{dt} = rN\left(1 - \frac{N}{K}\right)$$

symbol		meaning	SI unit
`N0`	in	initial population	—
`K`	in	carrying capacity (same unit)	—
`r`	in	intrinsic growth rate	1/s
`t`	in	elapsed time	s
`N`	out	population at time t	—
`dNdt`	out	growth rate at time t in cells per second	—
`dNdt_per_h`	out	growth rate at time t in cells per hour	—
`t_inflection`	out	time when N = K/2 (negative if already past)	s
`t_inflection_h`	out	time of the inflection point in hours	—

Applies when: 0 < N0 < K; r > 0; t >= 0.

```
$ cell-molecular-biology solve cmb_logistic_growth N0=10000 K=100000000 "r=0.72 1/h" "t=10 h"
N = 11813193.49
dNdt = 2083.535616
dNdt_per_h = 7500728.219
t_inflection = 46051.20183 s
t_inflection_h = 12.79200051
  by: Logistic growth of a population  (N = K / (1 + ((K - N0)/N0) e^(-r t)); dN/dt = r N (1 -
N/K); t_infl = ln((K - N0)/N0)/r)
  N0 = 10000
  K = 100000000
  r = 0.0002 1/s
  t = 36000 s
  assumes: 0 < N0 < K; r > 0; t >= 0
```

In the worked example, N0 = 10000, K = 100000000, r = 0.72 1/h, t = 10 h go through the equation above and give N = 11813193.49, dNdt = 2083.535616, dNdt_per_h = 7500728.219, t_inflection = 46051.20183 s. It applies only when 0 < N0 < K; r > 0; t >= 0: outside that it says what is wrong instead of returning a number.

Monod specific growth rate — `cmb_monod_growth_rate`

The specific growth rate of a microbial culture limited by one substrate (Monod), its fraction of the maximum and the corresponding doubling time.

$$\mu = \mu_{max} \frac{S}{K_s + S}$$

symbol		meaning	SI unit
`mu_max`	in	maximum specific growth rate	1/s
`S`	in	limiting substrate concentration	kg/m^3
`Ks`	in	half-saturation constant	kg/m^3
`mu`	out	specific growth rate	1/s
`mu_per_h`	out	growth rate per hour	—
`fraction`	out	mu / mu_max	—
`td_h`	out	doubling time ln2/mu in hours (0 when mu = 0)	—

Applies when: mu_max, Ks > 0; S >= 0; a single limiting substrate.

```
$ cell-molecular-biology solve cmb_monod_growth_rate "mu_max=0.72 1/h" "S=50 mg/L" "Ks=10 mg/L"
mu = 0.0001666666667 1/s
mu_per_h = 0.6
fraction = 0.8333333333
td_h = 1.155245301
  by: Monod specific growth rate  (mu = mu_max S / (Ks + S))
  mu_max = 0.0002 1/s
  S = 0.05 kg/m^3
  Ks = 0.01 kg/m^3
  assumes: mu_max, Ks > 0; S >= 0; a single limiting substrate
```

In the worked example, mu_max = 0.72 1/h, S = 50 mg/L, Ks = 10 mg/L go through the equation above and give mu = 0.0001666666667 1/s, mu_per_h = 0.6, fraction = 0.8333333333, td_h = 1.155245301. It applies only

when mu_max, Ks > 0; S >= 0; a single limiting substrate: outside that it says what is wrong instead of returning a number.

Number of molecules in a cell from a concentration — `cmb_molecules_per_cell`

The number of molecules of a species at concentration c in a compartment of volume V, and the concentration that one molecule per compartment represents.

$$N = c\,V\,N_A$$

symbol		meaning	SI unit
`c`	in	concentration	mol/m^3
`V`	in	cell (compartment) volume	m^3
`N`	out	number of molecules	—
`one_molecule`	out	concentration of one molecule in V	mol/m^3
`one_molecule_nM`	out	one molecule per compartment in nM	—

Applies when: c, V > 0.

```
$ cell-molecular-biology solve cmb_molecules_per_cell "c=1 (nmol/L)" "V=1 fL"
N = 0.602214076
one_molecule = 1.660539067e-06 mol/m^3
one_molecule_nM = 1.660539067
  by: Number of molecules in a cell from a concentration  (N = c V N_A)
  c = 1e-06 mol/m^3
  V = 1e-18 m^3
  assumes: c, V > 0
```

In the worked example, c = 1 (nmol/L), V = 1 fL go through the equation above and give N = 0.602214076, one_molecule = 1.660539067e-06 mol/m^3, one_molecule_nM = 1.660539067. It applies only when c, V > 0: outside that it says what is wrong instead of returning a number.

Specific growth rate fitted to growth-curve data — `cmb_growth_rate_fit`

The specific growth rate, doubling time and back-extrapolated initial count from counts (or OD) measured during exponential growth, by a least-squares line through ln N against time.

$$\ln N = \ln N_0 + \mu t, \quad t_d = \frac{\ln 2}{\mu}$$

symbol		meaning	SI unit
`t`	in	sampling times in hours	—
`N`	in	cell counts or optical densities (positive)	—
`mu_per_h`	out	specific growth rate per hour	—
`td_h`	out	doubling time in hours	—
`N0_fit`	out	fitted count at t = 0	—

symbol		meaning	SI unit
`r2`	out	coefficient of determination of the log-linear fit	—

Applies when: at least 3 points; counts > 0, increasing; exponential phase only.

```
$ cell-molecular-biology solve cmb_growth_rate_fit "t=[0, 1, 2, 3, 4]" "N=[1000, 1980, 4100, 7900,
16200]"
mu_per_h = 0.6953788399
td_h = 0.9967907287
N0_fit = 998.4351099
r2 = 0.9998073985
  by: Specific growth rate fitted to growth-curve data  (least-squares line ln N = ln N0 + mu t; td
= ln 2 / mu)
  assumes: at least 3 points; counts > 0, increasing; exponential phase only
```

In the worked example, t = [0, 1, 2, 3, 4], N = [1000, 1980, 4100, 7900, 16200] go through the equation above and give mu_per_h = 0.6953788399, td_h = 0.9967907287, N0_fit = 998.4351099, r2 = 0.9998073985. It applies only when at least 3 points; counts > 0, increasing; exponential phase only: outside that it says what is wrong instead of returning a number.

Time for a logistic population to reach a size — `cmb_logistic_time_to_reach`

The time for a population growing logistically from N0 (carrying capacity K, rate r) to reach N.

$$t = \frac{1}{r} \ln \frac{N(K - N_0)}{N_0(K - N)}$$

symbol		meaning	SI unit
`N0`	in	initial population	—
`K`	in	carrying capacity	—
`r`	in	intrinsic growth rate	1/s
`N`	in	target population	—
`t`	out	time to reach N	s
`t_h`	out	time in hours	—

Applies when: 0 < N0 < N < K; r > 0.

```
$ cell-molecular-biology solve cmb_logistic_time_to_reach N0=10000 K=100000000 "r=0.72 1/h"
N=50000000
t = 46051.20183 s
t_h = 12.79200051
  by: Time for a logistic population to reach a size  (t = (1/r) ln[ N (K - N0) / (N0 (K - N)) ])
  N0 = 10000
  K = 100000000
  r = 0.0002 1/s
  N = 50000000
  assumes: 0 < N0 < N < K; r > 0
```

In the worked example, N0 = 10000, K = 100000000, r = 0.72 1/h, N = 50000000 go through the equation above and give t = 46051.20183 s, t_h = 12.79200051. It applies only when 0 < N0 < N < K; r > 0: outside that it says what is wrong instead of returning a number.

Transformation efficiency — `cmb_transformation_efficiency`

Transformants per microgram of DNA from the colonies on a plate, the DNA mass used and the fraction of the transformation plated.

$$\eta = \frac{n}{m_{DNA}\,(V_{plated}/V_{total})}$$

symbol		meaning	SI unit
`colonies`	in	transformant colonies counted	—
`m_DNA`	in	DNA mass used in the transformation	kg
`V_plated`	in	volume plated	m^3
`V_total`	in	total volume after recovery	m^3
`efficiency`	out	transformants per microgram of DNA	—
`log10_efficiency`	out	log10 of the efficiency	—
`DNA_plated_ug`	out	micrograms of DNA plated	—

Applies when: colonies a whole number; m_DNA, volumes > 0; V_plated <= V_total.

```
$ cell-molecular-biology solve cmb_transformation_efficiency colonies=200 "m_DNA=1 ng"
"V_plated=100 uL" "V_total=1000 uL"
efficiency = 2000000
log10_efficiency = 6.301029996
DNA_plated_ug = 0.0001
  by: Transformation efficiency  (efficiency = colonies / (ug DNA x V_plated / V_total))
  m_DNA = 1e-12 kg
  V_plated = 1e-07 m^3
  V_total = 1e-06 m^3
  assumes: colonies a whole number; m_DNA, volumes > 0; V_plated <= V_total
```

In the worked example, colonies = 200, m_DNA = 1 ng, V_plated = 100 uL, V_total = 1000 uL go through the equation above and give efficiency = 2000000, log10_efficiency = 6.301029996, DNA_plated_ug = 0.0001. It applies only when colonies a whole number; m_DNA, volumes > 0; V_plated <= V_total: outside that it says what is wrong instead of returning a number.

Volume, surface area and surface-to-volume ratio of a spherical cell — `cmb_cell_geometry`

The volume (fL), surface area and surface-to-volume ratio of a spherical cell of a given diameter.

$$V = \tfrac{4}{3}\pi r^3, \quad A = 4\pi r^2, \quad \frac{A}{V} = \frac{3}{r}$$

symbol		meaning	SI unit
`d`	in	cell diameter	m

symbol		meaning	SI unit
`V`	out	cell volume	m^3
`V_fL`	out	volume in femtolitres (um^3)	—
`A`	out	surface area	m^2
`A_um2`	out	surface area in um^2	—
`SA_V_per_um`	out	surface-to-volume ratio in 1/um	—

Applies when: d > 0; a sphere.

```
$ cell-molecular-biology solve cmb_cell_geometry "d=10 um"
V = 5.235987756e-16 m^3
V_fL = 523.5987756
A = 3.141592654e-10 m^2
A_um2 = 314.1592654
SA_V_per_um = 0.6
  by: Volume, surface area and surface-to-volume ratio of a spherical cell  (V = (4/3) pi r^3, A =
4 pi r^2, A/V = 3/r)
  d = 1e-05 m
  assumes: d > 0; a sphere
```

In the worked example, d = 10 um go through the equation above and give V = 5.235987756e-16 m^3, V_fL = 523.5987756, A = 3.141592654e-10 m^2, A_um2 = 314.1592654. It applies only when d > 0; a sphere: outside that it says what is wrong instead of returning a number.

DNA and RNA (20)

A260/A280 and A260/A230 purity ratios — `cmb_absorbance_ratios`

The A260/A280 and A260/A230 ratios of a nucleic acid sample after subtracting the A320 background (about 1.8 for pure DNA, 2.0 for pure RNA; lower values suggest protein or organic contaminants).

$$\frac{A_{260} - A_{320}}{A_{280} - A_{320}}$$

symbol		meaning	SI unit
`A260`	in	absorbance at 260 nm	—
`A280`	in	absorbance at 280 nm	—
`A230`	in	absorbance at 230 nm (optional) (optional)	—
`A320`	in	background absorbance at 320 nm (default 0) (optional)	—
`ratio_260_280`	out	A260/A280 after background subtraction	—
`ratio_260_230`	out	A260/A230 after background subtraction (when A230 is given)	—

Applies when: absorbances > 0; A320 below the other readings.

```
$ cell-molecular-biology solve cmb_absorbance_ratios A260=0.9 A280=0.5 A230=0.42
ratio_260_280 = 1.8
ratio_260_230 = 2.142857143
  by: A260/A280 and A260/A230 purity ratios  (ratio = (A260 - A320)/(A280 - A320); (A260 -
A320)/(A230 - A320))
  A260 = 0.9
  A280 = 0.5
  A230 = 0.42
  assumes: absorbances > 0; A320 below the other readings
```

In the worked example, A260 = 0.9, A280 = 0.5, A230 = 0.42 go through the equation above and give ratio_260_280 = 1.8, ratio_260_230 = 2.142857143. It applies only when absorbances > 0; A320 below the other readings: outside that it says what is wrong instead of returning a number.

Basic GC melting temperature of an oligonucleotide — `cmb_tm_basic`

The basic melting temperature of a DNA oligonucleotide of 14 nt or more, Tm = 64.9 + 41 (G + C - 16.4) / N (50 mM Na+ assumed, no salt term).

$$T_m = 64.9 + 41\,\frac{n_G + n_C - 16.4}{N}$$

symbol		meaning	SI unit
`seq`	in	DNA sequence (A, C, G, T), at least 14 nt	—
`Tm`	out	melting temperature in degC	—
`gc_percent`	out	GC content in per cent	—
`length`	out	length in nt	—

Applies when: DNA letters only; N >= 14.

```
$ cell-molecular-biology solve cmb_tm_basic seq=AGCGGATAACAATTTCACACAGGA
Tm = 53.96666667
gc_percent = 41.66666667
length = 24
  by: Basic GC melting temperature of an oligonucleotide  (Tm = 64.9 + 41 (nG + nC - 16.4) / N)
  seq = AGCGGATAACAATTTCACACAGGA
  assumes: DNA letters only; N >= 14
```

In the worked example, seq = AGCGGATAACAATTTCACACAGGA go through the equation above and give Tm = 53.96666667, gc_percent = 41.66666667, length = 24. It applies only when DNA letters only; N >= 14: outside that it says what is wrong instead of returning a number.

Copy number and moles of a DNA or RNA molecule from its mass — `cmb_dna_copy_number`

The molecular weight, picomoles and number of copies in a mass of dsDNA, ssDNA or RNA of a given length (NEB average-mass formulas).

$$N = \frac{m}{M}\,N_A$$

symbol		meaning	SI unit
`m`	in	mass of nucleic acid	kg
`n`	in	length in base pairs (dsDNA) or bases (ssDNA, RNA)	—
`molecule`	in	dsDNA, ssDNA or RNA	—
`MW`	out	molecular weight	kg/mol
`MW_g_mol`	out	molecular weight in g/mol	—
`pmol`	out	picomoles	—
`copies`	out	number of molecules	—

Applies when: m > 0; n a whole number >= 1.

```
$ cell-molecular-biology solve cmb_dna_copy_number "m=1 ng" n=3000 molecule=dsDNA
MW = 1853.91604 kg/mol
MW_g_mol = 1853916.04
pmol = 0.0005393987529
copies = 324833521.6
  by: Copy number and moles of a DNA or RNA molecule from its mass  (MW = 617.96 n + 36.04 (dsDNA),
308.97 n + 18.02 (ssDNA), 320.5 n + 159.0 (RNA) g/mol; copies = m / MW x N_A)
  m = 1e-12 kg
  molecule = dsDNA
  assumes: m > 0; n a whole number >= 1
```

In the worked example, m = 1 ng, n = 3000, molecule = dsDNA go through the equation above and give MW = 1853.91604 kg/mol, MW_g_mol = 1853916.04, pmol = 0.0005393987529, copies = 324833521.6. It applies only when m > 0; n a whole number >= 1: outside that it says what is wrong instead of returning a number.

Extinction coefficient of a DNA oligonucleotide at 260 nm (nearest neighbour) — `cmb_oligo_extinction_coefficient`

The molar extinction coefficient at 260 nm of a single-stranded DNA oligonucleotide by the nearest-neighbour method, with micrograms and nanomoles per A260 unit (5'-hydroxyl oligo, 1 mL, 1 cm).

$$\varepsilon_{260} = \sum_{i=1}^{N-1} \varepsilon_{i,i+1} - \sum_{i=2}^{N-1} \varepsilon_i$$

symbol		meaning	SI unit
`seq`	in	DNA oligonucleotide 5'->3' (A, C, G, T)	—
`eps`	out	molar extinction coefficient at 260 nm	m^2/mol
`eps_L_mol_cm`	out	extinction coefficient in L/(mol cm)	—
`ug_per_A260`	out	micrograms per A260 unit (per mL at 1 cm)	—
`nmol_per_A260`	out	nanomoles per A260 unit (per mL at 1 cm)	—

Applies when: DNA letters only; single strand.

```
$ cell-molecular-biology solve cmb_oligo_extinction_coefficient seq=ATGCATGCAT
eps = 9800 m^2/mol
eps_L_mol_cm = 98000
ug_per_A260 = 30.8878561
nmol_per_A260 = 10.20408163
  by: Extinction coefficient of a DNA oligonucleotide at 260 nm (nearest neighbour)  (eps = sum of
nearest-neighbour pair values - sum of internal single-base values (Cantor-Warshaw-Shapiro table))
  seq = ATGCATGCAT
  assumes: DNA letters only; single strand
```

In the worked example, seq = ATGCATGCAT go through the equation above and give eps = 9800 m^2/mol, eps_L_mol_cm = 98000, ug_per_A260 = 30.8878561, nmol_per_A260 = 10.20408163. It applies only when DNA letters only; single strand: outside that it says what is wrong instead of returning a number.

GC content of a DNA or RNA sequence — `cmb_gc_content`

The fraction and percentage of G + C in a DNA or RNA sequence, with the base counts and the length.

$$\mathrm{GC} = \frac{n_G + n_C}{N}$$

symbol		meaning	SI unit
`seq`	in	DNA or RNA sequence (A, C, G, T or U)	—
`gc_fraction`	out	fraction of G + C	—
`gc_percent`	out	GC content in per cent	—
`length`	out	length in bases	—
`G`	out	number of G	—
`C`	out	number of C	—
`A`	out	number of A	—
`T_or_U`	out	number of T (DNA) or U (RNA)	—

Applies when: only A, C, G, T or U.

```
$ cell-molecular-biology solve cmb_gc_content seq=ATGCGCGATTACGC
gc_fraction = 0.5714285714
gc_percent = 57.14285714
length = 14
G = 4
C = 4
A = 3
T_or_U = 3
  by: GC content of a DNA or RNA sequence  (GC = (n_G + n_C) / N)
  seq = ATGCGCGATTACGC
  assumes: only A, C, G, T or U
```

In the worked example, seq = ATGCGCGATTACGC go through the equation above and give gc_fraction = 0.5714285714, gc_percent = 57.14285714, length = 14, G = 4. It applies only when only A, C, G, T or U: outside that it says what is wrong instead of returning a number.

Insert mass for a ligation at an insert:vector molar ratio — `cmb_ligation_insert_mass`

The mass of insert DNA needed for a ligation at a chosen insert:vector molar ratio, and the femtomoles of vector and insert (660 g/mol per bp).

$$m_{ins} = r\, m_{vec}\, \frac{L_{ins}}{L_{vec}}$$

symbol		meaning	SI unit
`m_vector`	in	mass of vector DNA	kg
`L_vector`	in	vector length in bp	—
`L_insert`	in	insert length in bp	—
`ratio`	in	insert:vector molar ratio (e.g. 3 for 3:1)	—
`m_insert`	out	mass of insert	kg
`m_insert_ng`	out	mass of insert in ng	—
`vector_fmol`	out	vector in fmol	—
`insert_fmol`	out	insert in fmol	—

Applies when: masses, ratio > 0; lengths whole numbers >= 1.

```
$ cell-molecular-biology solve cmb_ligation_insert_mass "m_vector=50 ng" L_vector=3000
L_insert=1000 ratio=3
m_insert = 5e-11 kg
m_insert_ng = 50
vector_fmol = 25.25252525
insert_fmol = 75.75757576
  by: Insert mass for a ligation at an insert:vector molar ratio  (m_insert = ratio x m_vector x
L_insert / L_vector)
  m_vector = 5e-11 kg
  ratio = 3
  assumes: masses, ratio > 0; lengths whole numbers >= 1
```

In the worked example, m_vector = 50 ng, L_vector = 3000, L_insert = 1000, ratio = 3 go through the equation above and give m_insert = 5e-11 kg, m_insert_ng = 50, vector_fmol = 25.25252525, insert_fmol = 75.75757576. It applies only when masses, ratio > 0; lengths whole numbers >= 1: outside that it says what is wrong instead of returning a number.

Longest open reading frame on both strands — `cmb_orf_finder`

The longest open reading frame (ATG to the first in-frame stop) in all six frames of a DNA sequence with at least min_aa codons, its strand, coordinates on that strand, protein and the number of such ORFs.

$$\mathrm{ORF} = \mathrm{ATG}\,(\mathrm{NNN})^{n}\,\{\mathrm{TAA}, \mathrm{TAG}, \mathrm{TGA}\}$$

symbol		meaning	SI unit
`seq`	in	DNA sequence 5'->3'	—
`min_aa`	in	minimum ORF length in codons (amino acids, stop excluded)	—

symbol		meaning	SI unit
`strand`	out	+ (given strand) or - (reverse complement)	—
`start`	out	1-based start of the ATG on that strand (read 5'->3')	—
`end`	out	1-based end of the stop codon on that strand	—
`length_nt`	out	ORF length in nt including the stop	—
`length_aa`	out	protein length in amino acids	—
`protein`	out	encoded protein (one-letter code)	—
`n_orfs`	out	number of ORFs of at least min_aa codons	—

Applies when: DNA letters only; min_aa >= 1; at least one ORF.

```
$ cell-molecular-biology solve cmb_orf_finder
seq=CCATGAAACGCATTAGCACCACCATTACCACCACCATCACCATTACCACAGGTAACGGTGCGGGCTGAGG min_aa=10
strand = +
start = 3
end = 68
length_nt = 66
length_aa = 21
protein = MKRISTTITTTTITITTGNGAG
n_orfs = 1
  by: Longest open reading frame on both strands  (scan the three frames of each strand; an ORF
starts at the first ATG after the previous stop and ends at the next stop; keep the longest (ties:
+ strand, then the lowest start))
  seq = CCATGAAACGCATTAGCACCACCATTACCACCACCATCACCATTACCACAGGTAACGGTGCGGGCTGAGG
  assumes: DNA letters only; min_aa >= 1; at least one ORF
```

In the worked example, seq = CCATGAAACGCATTAGCACCACCATTACCACCACCATCACCATTACCACAGGTAACGGTGCGGGCTGAGG, min_aa = 10 go through the equation above and give strand = +, start = 3, end = 68, length_nt = 66. It applies only when DNA letters only; min_aa >= 1; at least one ORF: outside that it says what is wrong instead of returning a number.

Molecular weight of a DNA or RNA oligonucleotide — `cmb_oligo_molecular_weight`

The average molecular weight of a single- or double-stranded DNA or RNA oligonucleotide, with a 5'-phosphate (sum of nucleotide monophosphates minus waters) and as the 5'-hydroxyl form of synthetic oligos.

$$M = \sum_i m_i - (n-1)\, M_{H_2O}$$

symbol		meaning	SI unit
`seq`	in	DNA or RNA sequence 5'->3'	—
`strands`	in	single strand or the duplex with its complement	—
`MW`	out	molecular weight (5'-phosphate)	kg/mol
`MW_g_mol`	out	molecular weight in g/mol (5'-phosphate)	—
`MW_hydroxyl_g_mol`	out	molecular weight in g/mol with 5'-hydroxyl ends	—

Applies when: only A, C, G, T (DNA) or A, C, G, U (RNA).

```
$ cell-molecular-biology solve cmb_oligo_molecular_weight seq=ATGCATGCAT strands=single
MW = 3.1069898 kg/mol
MW_g_mol = 3106.9898
MW_hydroxyl_g_mol = 3027.009898
  by: Molecular weight of a DNA or RNA oligonucleotide  (MW = sum of nucleotide monophosphate
weights - (n - 1) x 18.0153; 5'-OH form minus HPO3 (79.98) per strand)
  seq = ATGCATGCAT
  strands = single
  assumes: only A, C, G, T (DNA) or A, C, G, U (RNA)
```

In the worked example, seq = ATGCATGCAT, strands = single go through the equation above and give MW = 3.1069898 kg/mol, MW_g_mol = 3106.9898, MW_hydroxyl_g_mol = 3027.009898. It applies only when only A, C, G, T (DNA) or A, C, G, U (RNA): outside that it says what is wrong instead of returning a number.

Mutation rate by the fluctuation-test p0 method — `cmb_mutation_rate_p0`

The expected number of mutations per culture m = -ln p0 from the fraction of parallel cultures without mutants (Luria-Delbruck p0 method) and the mutation rate m / Nt.

$$p_0 = e^{-m}, \quad \mu = \frac{-\ln p_0}{N_t}$$

symbol		meaning	SI unit
`n0`	in	number of cultures with no mutants	—
`n`	in	total number of cultures	—
`Nt`	in	final number of cells per culture	—
`p0`	out	fraction of cultures without mutants	—
`m`	out	mean mutations per culture	—
`mutation_rate`	out	mutations per cell per generation (m / Nt)	—

Applies when: 1 <= n0 < n; Nt > 0; Poisson number of mutational events per culture.

```
$ cell-molecular-biology solve cmb_mutation_rate_p0 n0=11 n=20 Nt=200000000
p0 = 0.55
m = 0.5978370008
mutation_rate = 2.989185004e-09
  by: Mutation rate by the fluctuation-test p0 method  (p0 = e^-m, so m = -ln(n0/n); mu = m / Nt)
  Nt = 200000000
  assumes: 1 <= n0 < n; Nt > 0; Poisson number of mutational events per culture
```

In the worked example, n0 = 11, n = 20, Nt = 200000000 go through the equation above and give p0 = 0.55, m = 0.5978370008, mutation_rate = 2.989185004e-09. It applies only when 1 <= n0 < n; Nt > 0; Poisson number of mutational events per culture: outside that it says what is wrong instead of returning a number.

Nearest-neighbour melting temperature of a DNA duplex — `cmb_tm_nearest_neighbor`

The melting temperature of a perfectly matched DNA duplex by nearest-neighbour thermodynamics (SantaLucia-Hicks 2004 unified parameters) with the SantaLucia (1998) sodium correction, and the duplex dH, dS and dG at 37 degC.

$$T_m = \frac{1000\,\Delta H^\circ}{\Delta S^\circ + 0.368(N-1)\ln[\mathrm{Na}^+] + R\ln k} - 273.15$$

symbol		meaning	SI unit
`seq`	in	DNA sequence 5'->3' (A, C, G, T)	—
`Na`	in	monovalent cation (Na+) concentration	mol/m^3
`c1`	in	concentration of the more concentrated strand	mol/m^3
`c2`	in	concentration of the less concentrated strand (0 if in large excess)	mol/m^3
`Tm`	out	melting temperature in degC	—
`dH`	out	duplex enthalpy in kcal/mol	—
`dS`	out	salt-corrected duplex entropy in cal/(mol K)	—
`dG37`	out	free energy at 37 degC in kcal/mol	—

Applies when: DNA letters, length >= 2; Na+ > 0; c1 > c2/2 (self-complementary: k = c1).

```
$ cell-molecular-biology solve cmb_tm_nearest_neighbor seq=CGTTCCAAAGATGTGGGCATGAGCTTAC "Na=50
(mmol/L)" "c1=250 (nmol/L)" "c2=250 (nmol/L)"
Tm = 62.58162841
dH = -221.7
dS = -628.7655959
dG37 = -26.68835044
  by: Nearest-neighbour melting temperature of a DNA duplex  (sum nearest-neighbour dH and dS with
initiation and terminal A/T terms (+ symmetry); dS += 0.368 (N-1) ln[Na+]; Tm = 1000 dH / (dS + R
ln k) - 273.15, k = c1 - c2/2)
  Na = 50 mol/m^3
  c1 = 0.00025 mol/m^3
  c2 = 0.00025 mol/m^3
  seq = CGTTCCAAAGATGTGGGCATGAGCTTAC
  assumes: DNA letters, length >= 2; Na+ > 0; c1 > c2/2 (self-complementary: k = c1)
```

In the worked example, seq = CGTTCCAAAGATGTGGGCATGAGCTTAC, Na = 50 (mmol/L), c1 = 250 (nmol/L), c2 = 250 (nmol/L) go through the equation above and give Tm = 62.58162841, dH = -221.7, dS = -628.7655959, dG37 = -26.68835044. It applies only when DNA letters, length >= 2; Na+ > 0; c1 > c2/2 (self-complementary: k = c1): outside that it says what is wrong instead of returning a number.

Nucleic acid concentration from A260 — `cmb_nucleic_acid_a260`

The concentration of dsDNA, ssDNA or RNA from its absorbance at 260 nm (50, 33 and 40 ug/mL per unit at 1 cm), with the dilution and path length, and the mass in a volume.

$$c = A_{260} \times F \times DF/\ell$$

symbol		meaning	SI unit
`A260`	in	absorbance at 260 nm (blank-corrected)	—
`molecule`	in	dsDNA, ssDNA or RNA	—
`dilution`	in	dilution factor of the measured sample (default 1) (optional)	—

symbol		meaning	SI unit
`l`	in	path length (default 1 cm) (optional)	m
`V`	in	volume of the stock (optional, for the total mass) (optional)	m^3
`c`	out	concentration of the stock	kg/m^3
`c_ug_mL`	out	concentration in ug/mL (= ng/uL)	—
`mass_ug`	out	micrograms in V (when V is given)	—

Applies when: A260 >= 0 (best within 0.1-1.0); dilution >= 1; l > 0.

```
$ cell-molecular-biology solve cmb_nucleic_acid_a260 A260=0.5 molecule=dsDNA dilution=10
c = 0.25 kg/m^3
c_ug_mL = 250
  by: Nucleic acid concentration from A260  (c (ug/mL) = A260 x factor x dilution / l(cm); factor
50 (dsDNA), 33 (ssDNA), 40 (RNA))
  A260 = 0.5
  dilution = 10
  molecule = dsDNA
  assumes: A260 >= 0 (best within 0.1-1.0); dilution >= 1; l > 0
```

In the worked example, A260 = 0.5, molecule = dsDNA, dilution = 10 go through the equation above and give c = 0.25 kg/m^3, c_ug_mL = 250. It applies only when A260 >= 0 (best within 0.1-1.0); dilution >= 1; l > 0: outside that it says what is wrong instead of returning a number.

Number of clones for a genomic library (Clarke-Carbon) — `cmb_library_size`

The number of random clones needed so that any given sequence is present with probability P, for a genome of size G and inserts of size I.

$$N = \frac{\ln(1 - P)}{\ln(1 - I/G)}$$

symbol		meaning	SI unit
`G`	in	genome size in bp	—
`I`	in	insert size in bp	—
`P`	in	desired probability (e.g. 99%)	—
`clones_exact`	out	number of clones (real value)	—
`clones`	out	number of clones rounded up	—

Applies when: 0 < I < G; 0 < P < 1.

```
$ cell-molecular-biology solve cmb_library_size G=3000000000 I=40000 P=0.99
clones_exact = 345385.4614
clones = 345386
  by: Number of clones for a genomic library (Clarke-Carbon)  (N = ln(1 - P) / ln(1 - I/G))
  G = 3000000000
  I = 40000
  P = 0.99
  assumes: 0 < I < G; 0 < P < 1
```

In the worked example, G = 3000000000, I = 40000, P = 0.99 go through the equation above and give clones_exact = 345385.4614, clones = 345386. It applies only when 0 < I < G; 0 < P < 1: outside that it says what is wrong instead of returning a number.

Primer melting temperature by the Wallace rule — `cmb_tm_wallace`

The approximate melting temperature (degC) of a short DNA primer (about 14-20 nt) by the Wallace rule, 2 degC per A/T and 4 degC per G/C.

$$T_m = 2(n_A + n_T) + 4(n_G + n_C)\ ^\circ\mathrm{C}$$

symbol		meaning	SI unit
`seq`	in	DNA primer sequence (A, C, G, T)	—
`Tm`	out	melting temperature in degC	—
`AT`	out	number of A and T	—
`GC`	out	number of G and C	—

Applies when: DNA letters only; at least 2 bases (meant for 14-20 nt).

```
$ cell-molecular-biology solve cmb_tm_wallace seq=ACGTTGCAATGCCGTA
Tm = 48
AT = 8
GC = 8
  by: Primer melting temperature by the Wallace rule  (Tm = 2 (A + T) + 4 (G + C) degC)
  seq = ACGTTGCAATGCCGTA
  assumes: DNA letters only; at least 2 bases (meant for 14-20 nt)
```

In the worked example, seq = ACGTTGCAATGCCGTA go through the equation above and give Tm = 48, AT = 8, GC = 8. It applies only when DNA letters only; at least 2 bases (meant for 14-20 nt): outside that it says what is wrong instead of returning a number.

Protein concentration by Warburg-Christian (A280 and A260) — `cmb_warburg_christian`

The protein concentration (mg/mL) of a sample containing nucleic acid from its absorbances at 280 and 260 nm, protein = 1.55 A280 - 0.76 A260 (1 cm).

$$c_{prot} = 1.55A_{280} - 0.76A_{260}\ \mathrm{mg/mL}$$

symbol		meaning	SI unit
`A280`	in	absorbance at 280 nm	—
`A260`	in	absorbance at 260 nm	—
`protein`	out	protein concentration	kg/m^3
`protein_mg_mL`	out	protein in mg/mL	—
`ratio_280_260`	out	A280/A260	—

Applies when: absorbances >= 0, 1 cm path; a positive estimate.

```
$ cell-molecular-biology solve cmb_warburg_christian A280=0.8 A260=0.5
protein = 0.86 kg/m^3
protein_mg_mL = 0.86
ratio_280_260 = 1.6
  by: Protein concentration by Warburg-Christian (A280 and A260)  (protein (mg/mL) = 1.55 A280 -
0.76 A260)
  A280 = 0.8
  A260 = 0.5
  assumes: absorbances >= 0, 1 cm path; a positive estimate
```

In the worked example, A280 = 0.8, A260 = 0.5 go through the equation above and give protein = 0.86 kg/m^3, protein_mg_mL = 0.86, ratio_280_260 = 1.6. It applies only when absorbances >= 0, 1 cm path; a positive estimate: outside that it says what is wrong instead of returning a number.

Protein concentration from A280 and the extinction coefficient — `cmb_protein_concentration_a280`

The molar (and, with the molar mass, mass) concentration of a protein from its absorbance at 280 nm and molar extinction coefficient (Beer-Lambert).

$$c = \frac{A_{280}}{\varepsilon \ell}$$

symbol		meaning	SI unit
`A280`	in	absorbance at 280 nm	—
`eps`	in	molar extinction coefficient at 280 nm	m^2/mol
`MW`	in	molar mass of the protein (optional) (optional)	kg/mol
`l`	in	path length (default 1 cm) (optional)	m
`c`	out	protein concentration	mol/m^3
`c_uM`	out	concentration in uM	—
`c_mg_mL`	out	concentration in mg/mL (when MW is given)	—

Applies when: A280 >= 0; eps, l > 0.

```
$ cell-molecular-biology solve cmb_protein_concentration_a280 A280=0.75 "eps=43570 L/(mol*cm)"
"MW=66430 g/mol"
c = 0.01721367914 mol/m^3
c_uM = 17.21367914
c_mg_mL = 1.143504705
  by: Protein concentration from A280 and the extinction coefficient  (c = A / (eps l); mass
concentration = c x MW)
  A280 = 0.75
  eps = 4357 m^2/mol
  MW = 66.43 kg/mol
  assumes: A280 >= 0; eps, l > 0
```

In the worked example, A280 = 0.75, eps = 43570 L/(mol*cm), MW = 66430 g/mol go through the equation above and give c = 0.01721367914 mol/m^3, c_uM = 17.21367914, c_mg_mL = 1.143504705. It applies only when A280 >= 0; eps, l > 0: outside that it says what is wrong instead of returning a number.

Reverse complement of a DNA or RNA sequence — `cmb_reverse_complement`

The reverse complement (the other strand read 5'->3'), the complement and the reverse of a DNA or RNA sequence.

$$\mathrm{rc}(s) = \overline{s_N} \cdots \overline{s_1}$$

symbol		meaning	SI unit
`seq`	in	DNA or RNA sequence 5'->3'	—
`reverse_complement`	out	reverse complement 5'->3'	—
`complement`	out	complement 3'->5'	—
`reverse`	out	the sequence reversed	—

Applies when: only A, C, G, T or U.

```
$ cell-molecular-biology solve cmb_reverse_complement seq=ATGCGTACCTGA
reverse_complement = TCAGGTACGCAT
complement = TACGCATGGACT
reverse = AGTCCATGCGTA
  by: Reverse complement of a DNA or RNA sequence  (complement A-T (A-U), G-C, then reverse)
  seq = ATGCGTACCTGA
  assumes: only A, C, G, T or U
```

In the worked example, seq = ATGCGTACCTGA go through the equation above and give reverse_complement = TCAGGTACGCAT, complement = TACGCATGGACT, reverse = AGTCCATGCGTA. It applies only when only A, C, G, T or U: outside that it says what is wrong instead of returning a number.

Salt-adjusted melting temperature of an oligonucleotide — `cmb_tm_salt_adjusted`

The melting temperature of a DNA oligonucleotide of 14 nt or more with a sodium correction: Tm = 81.5 + 16.6 log10[Na+] + 0.41 %GC - 600/N.

$$T_m = 81.5 + 16.6 \log_{10}[\mathrm{Na}^+] + 0.41\,\%GC - \frac{600}{N}$$

symbol		meaning	SI unit
`seq`	in	DNA sequence (A, C, G, T), at least 14 nt	—
`Na`	in	monovalent cation (Na+) concentration	mol/m^3
`Tm`	out	melting temperature in degC	—
`gc_percent`	out	GC content in per cent	—
`salt_term`	out	salt term 16.6 log10[Na+] in degC	—

Applies when: DNA letters only; N >= 14; Na+ > 0.

```
$ cell-molecular-biology solve cmb_tm_salt_adjusted seq=AGCGGATAACAATTTCACACAGGA "Na=50 (mmol/L)"
Tm = 51.98623541
gc_percent = 41.66666667
salt_term = -21.59709793
  by: Salt-adjusted melting temperature of an oligonucleotide  (Tm = 81.5 + 16.6 log10([Na+]/M) +
0.41 %GC - 600/N)
  Na = 50 mol/m^3
  seq = AGCGGATAACAATTTCACACAGGA
  assumes: DNA letters only; N >= 14; Na+ > 0
```

In the worked example, seq = AGCGGATAACAATTTCACACAGGA, Na = 50 (mmol/L) go through the equation above and give Tm = 51.98623541, gc_percent = 41.66666667, salt_term = -21.59709793. It applies only when DNA letters only; N >= 14; Na+ > 0: outside that it says what is wrong instead of returning a number.

Sequencing coverage and uncovered fraction (Lander-Waterman) — `cmb_genome_coverage`

The mean sequencing depth C = L N / G, the expected fraction and number of bases not covered (e^-C) and the expected number of contigs (N e^-C).

$$C = \frac{LN}{G}, \quad P_0 = e^{-C}$$

symbol		meaning	SI unit
`L`	in	read length in bp	—
`N`	in	number of reads	—
`G`	in	genome size in bp	—
`coverage`	out	mean coverage (depth)	—
`fraction_uncovered`	out	fraction of bases not covered e^-C	—
`bases_uncovered`	out	expected bases not covered	—
`contigs`	out	expected number of contigs N e^-C	—

Applies when: L, N whole numbers >= 1; G > 0; reads placed uniformly at random.

```
$ cell-molecular-biology solve cmb_genome_coverage L=150 N=2000000 G=5000000
coverage = 60
fraction_uncovered = 8.756510763e-27
bases_uncovered = 4.378255381e-20
contigs = 1.751302153e-20
  by: Sequencing coverage and uncovered fraction (Lander-Waterman)  (C = L N / G; P(0 reads) = e^-C
(Poisson))
  G = 5000000
  assumes: L, N whole numbers >= 1; G > 0; reads placed uniformly at random
```

In the worked example, L = 150, N = 2000000, G = 5000000 go through the equation above and give coverage = 60, fraction_uncovered = 8.756510763e-27, bases_uncovered = 4.378255381e-20, contigs = 1.751302153e-20. It applies only when L, N whole numbers >= 1; G > 0; reads placed uniformly at random: outside that it says what is wrong instead of returning a number.

Transcription of a DNA strand into mRNA — `cmb_transcribe`

The mRNA (5'->3') transcribed from a DNA sequence given 5'->3' as either the coding (sense) strand or the template (antisense) strand.

$$\mathrm{mRNA} = \mathrm{coding}(T \to U) = \mathrm{rc}(\mathrm{template})(T \to U)$$

symbol		meaning	SI unit
`seq`	in	DNA sequence 5'->3'	—
`strand`	in	whether the given strand is the coding or the template strand	—
`mRNA`	out	mRNA sequence 5'->3'	—
`length`	out	length in nt	—

Applies when: DNA letters only.

```
$ cell-molecular-biology solve cmb_transcribe seq=ATGGCCTTTAAG strand=coding
mRNA = AUGGCCUUUAAG
length = 12
  by: Transcription of a DNA strand into mRNA  (coding strand: T -> U; template strand: reverse
complement, then T -> U)
  strand = coding
  seq = ATGGCCTTTAAG
  assumes: DNA letters only
```

In the worked example, seq = ATGGCCTTTAAG, strand = coding go through the equation above and give mRNA = AUGGCCUUUAAG, length = 12. It applies only when DNA letters only: outside that it says what is wrong instead of returning a number.

Translation of an mRNA or DNA sequence into protein — `cmb_translate`

The protein encoded from the first base (frame 1) by the standard genetic code: up to the first stop codon, and the full translation with stops as *. A trailing incomplete codon is ignored.

$$\mathrm{codon} \mapsto \mathrm{amino\ acid}\ (\mathrm{table}\ 1)$$

symbol		meaning	SI unit
`seq`	in	mRNA or coding-strand DNA 5'->3'	—
`protein`	out	amino acids up to the first stop (one-letter code)	—
`full_translation`	out	full translation with * for stops	—
`length_aa`	out	length of the protein up to the first stop	—
`stops`	out	number of stop codons	—

Applies when: only A, C, G, T or U; at least one codon.

```
$ cell-molecular-biology solve cmb_translate seq=AUGGCCAUUGUAAUGGGCCGCUGAAAGGGUGCCCGAUAG
protein = MAIVMGR
full_translation = MAIVMGR*KGAR*
length_aa = 7
stops = 2
  by: Translation of an mRNA or DNA sequence into protein  (read codons from base 1 with the
standard code (NCBI table 1))
  seq = AUGGCCAUUGUAAUGGGCCGCUGAAAGGGUGCCCGAUAG
  assumes: only A, C, G, T or U; at least one codon
```

In the worked example, seq = AUGGCCAUUGUAAUGGGCCGCUGAAAGGGUGCCCGAUAG go through the equation above and give protein = MAIVMGR, full_translation = MAIVMGR*KGAR*, length_aa = 7, stops = 2. It applies only when only A, C, G, T or U; at least one codon: outside that it says what is wrong instead of returning a number.

Enzyme kinetics (12)

Competitive inhibition rate — `cmb_competitive_inhibition`

The rate of an enzyme with a competitive inhibitor (binding free enzyme only): Km rises by (1 + I/Ki), Vmax unchanged.

$$v = \frac{V_{max}[S]}{K_m(1 + [I]/K_i) + [S]}$$

symbol		meaning	SI unit
`Vmax`	in	maximum rate without inhibitor	mol/(m^3*s)
`Km`	in	Michaelis constant	mol/m^3
`S`	in	substrate concentration	mol/m^3
`I`	in	inhibitor concentration	mol/m^3
`Ki`	in	inhibition (dissociation) constant	mol/m^3
`v`	out	rate with the inhibitor	mol/(m^3*s)
`Km_app`	out	apparent Km	mol/m^3
`Vmax_app`	out	apparent Vmax	mol/(m^3*s)
`percent_inhibition`	out	per cent inhibition at this S	—

Applies when: Vmax, Km, Ki > 0; S, I >= 0; rapid-equilibrium inhibitor binding.

```
$ cell-molecular-biology solve cmb_competitive_inhibition "Vmax=6 (umol/L)/min" "Km=50 (umol/L)"
"S=100 (umol/L)" "I=20 (umol/L)" "Ki=10 (umol/L)"
v = 4e-05 mol/(m^3*s)
Km_app = 0.15 mol/m^3
Vmax_app = 0.0001 mol/(m^3*s)
percent_inhibition = 40
  by: Competitive inhibition rate  (v = Vmax S / (Km (1 + I/Ki) + S))
  Vmax = 0.0001 mol/(m^3*s)
  Km = 0.05 mol/m^3
  S = 0.1 mol/m^3
  I = 0.02 mol/m^3
  Ki = 0.01 mol/m^3
  assumes: Vmax, Km, Ki > 0; S, I >= 0; rapid-equilibrium inhibitor binding
```

In the worked example, Vmax = 6 (umol/L)/min, Km = 50 (umol/L), S = 100 (umol/L), I = 20 (umol/L), Ki = 10 (umol/L) go through the equation above and give v = 4e-05 mol/(m^3*s), Km_app = 0.15 mol/m^3, Vmax_app = 0.0001 mol/(m^3*s), percent_inhibition = 40. It applies only when Vmax, Km, Ki > 0; S, I >= 0; rapid-equilibrium inhibitor binding: outside that it says what is wrong instead of returning a number.

Enzyme activity and specific activity from a spectrophotometric assay — `cmb_enzyme_assay_activity`

The enzyme activity (units = umol/min) in a cuvette from the absorbance change over a time (Beer-Lambert), the activity per mL of the enzyme sample and, with its protein concentration, the specific activity (U/mg).

$$\mathrm{U/mL} = \frac{(\Delta A/\Delta t)\,V_{assay}}{\varepsilon\,\ell\,V_{enz}}$$

symbol		meaning	SI unit
`dA`	in	absorbance change over dt	—
`dt`	in	time of the absorbance change	s
`eps`	in	molar absorptivity of the measured product or substrate (NADH 6220 L/(mol cm))	m^2/mol
`l`	in	path length	m
`V_assay`	in	total assay volume	m^3
`V_enzyme`	in	volume of enzyme sample added	m^3
`protein`	in	protein concentration of the enzyme sample (optional) (optional)	kg/m^3
`rate`	out	rate of concentration change in the cuvette	mol/(m^3*s)
`units`	out	activity in the cuvette in umol/min (U)	—
`units_per_mL`	out	activity of the enzyme sample in U/mL	—
`specific_activity`	out	specific activity in U/mg (when protein is given)	—

Applies when: dA != 0; dt, eps, l, volumes > 0; V_enzyme <= V_assay; linear initial rate.

```
$ cell-molecular-biology solve cmb_enzyme_assay_activity dA=0.12 "dt=1 min" "eps=6220 L/(mol*cm)"
"l=1 cm" "V_assay=1 mL" "V_enzyme=20 uL" "protein=0.5 mg/mL"
rate = 0.0003215434084 mol/(m^3*s)
units = 0.0192926045
units_per_mL = 0.9646302251
specific_activity = 1.92926045
  by: Enzyme activity and specific activity from a spectrophotometric assay  (rate = |dA|/dt / (eps
l); U = rate x V_assay (umol/min); U/mL = U / V_enzyme; U/mg = (U/mL) / (mg/mL))
  dA = 0.12
  dt = 60 s
  eps = 622 m^2/mol
  l = 0.01 m
  V_assay = 1e-06 m^3
  V_enzyme = 2e-08 m^3
  protein = 0.5 kg/m^3
  assumes: dA != 0; dt, eps, l, volumes > 0; V_enzyme <= V_assay; linear initial rate
```

In the worked example, dA = 0.12, dt = 1 min, eps = 6220 L/(mol*cm), l = 1 cm, V_assay = 1 mL, V_enzyme = 20 uL, protein = 0.5 mg/mL go through the equation above and give rate = 0.0003215434084 mol/(m^3*s), units = 0.0192926045, units_per_mL = 0.9646302251, specific_activity = 1.92926045. It applies only when dA != 0; dt, eps, l, volumes > 0; V_enzyme <= V_assay; linear initial rate: outside that it says what is wrong instead of returning a number.

Inhibition constant from IC50 (Cheng-Prusoff) — `cmb_ic50_to_ki`

The inhibition constant Ki from a measured IC50 at substrate concentration S for a competitive, uncompetitive or noncompetitive inhibitor (Cheng-Prusoff relations).

$$K_i = \frac{IC_{50}}{1 + [S]/K_m} \ \text{(competitive)}$$

symbol		meaning	SI unit
`mode`	in	mechanism of inhibition	—
`IC50`	in	inhibitor concentration giving 50 % inhibition	mol/m^3
`S`	in	substrate concentration of the assay	mol/m^3
`Km`	in	Michaelis constant	mol/m^3
`Ki`	out	inhibition constant	mol/m^3
`Ki_uM`	out	Ki in uM	—

Applies when: IC50, S, Km > 0; inhibitor not depleted (I >> E).

```
$ cell-molecular-biology solve cmb_ic50_to_ki mode=competitive "IC50=10 (umol/L)" "S=100 (umol/L)"
"Km=50 (umol/L)"
Ki = 0.003333333333 mol/m^3
Ki_uM = 3.333333333
  by: Inhibition constant from IC50 (Cheng-Prusoff)  (competitive Ki = IC50/(1 + S/Km);
uncompetitive Ki = IC50/(1 + Km/S); noncompetitive Ki = IC50)
  IC50 = 0.01 mol/m^3
  S = 0.1 mol/m^3
  Km = 0.05 mol/m^3
  mode = competitive
  assumes: IC50, S, Km > 0; inhibitor not depleted (I >> E)
```

In the worked example, mode = competitive, IC50 = 10 (umol/L), S = 100 (umol/L), Km = 50 (umol/L) go through the equation above and give Ki = 0.003333333333 mol/m^3, Ki_uM = 3.333333333. It applies only when IC50, S, Km > 0; inhibitor not depleted (I >> E): outside that it says what is wrong instead of returning a number.

Km and Vmax by nonlinear least squares — `cmb_michaelis_menten_nonlinear_fit`

Km and Vmax fitted directly to initial-rate data by nonlinear least squares on v = Vmax S/(Km + S) (no linearizing transform), with the residual sum of squares and r^2.

$$\min_{V,K} \sum_i \left(v_i - \frac{V S_i}{K + S_i} \right)^2$$

symbol		meaning	SI unit
`S`	in	substrate concentrations (any one unit)	—
`v`	in	initial rates (any one unit)	—
`Km`	out	Michaelis constant (unit of S)	—
`Vmax`	out	maximum rate (unit of v)	—
`rss`	out	residual sum of squares	—
`r2`	out	coefficient of determination	—

Applies when: at least 3 points; S, v > 0; an interior least-squares minimum.

```
$ cell-molecular-biology solve cmb_michaelis_menten_nonlinear_fit "S=[1, 2, 5, 10, 20, 50]" "v=[16,
29.5, 49, 68, 79, 92]"
Km = 5.079276612
Vmax = 100.7136844
rss = 5.867526413
r2 = 0.9986415273
  by: Km and Vmax by nonlinear least squares  (Vmax profiled out (Vmax = sum v g / sum g^2, g =
S/(Km + S)); the stationary point in Km found by bisection)
  assumes: at least 3 points; S, v > 0; an interior least-squares minimum
```

In the worked example, S = [1, 2, 5, 10, 20, 50], v = [16, 29.5, 49, 68, 79, 92] go through the equation above and give Km = 5.079276612, Vmax = 100.7136844, rss = 5.867526413, r2 = 0.9986415273. It applies only when at least 3 points; S, v > 0; an interior least-squares minimum: outside that it says what is wrong instead of returning a number.

Km and Vmax from a Lineweaver-Burk plot — `cmb_lineweaver_burk_fit`

Km and Vmax from initial-rate data by the double-reciprocal (Lineweaver-Burk) least-squares line 1/v = (Km/Vmax)(1/S) + 1/Vmax.

$$\frac{1}{v} = \frac{K_m}{V_{max}} \frac{1}{[S]} + \frac{1}{V_{max}}$$

symbol		meaning	SI unit
`S`	in	substrate concentrations (any one unit, e.g. uM)	—
`v`	in	initial rates (any one unit)	—
`Km`	out	Michaelis constant (unit of S)	—
`Vmax`	out	maximum rate (unit of v)	—
`slope`	out	slope Km/Vmax	—
`intercept`	out	intercept 1/Vmax	—
`r2`	out	r^2 of the double-reciprocal line	—

Applies when: at least 3 points; S, v > 0; positive slope and intercept.

```
$ cell-molecular-biology solve cmb_lineweaver_burk_fit "S=[1, 2, 5, 10, 20, 50]" "v=[16.7, 28.6,
50, 66.7, 80, 90.9]"
Km = 4.982268904
Vmax = 99.88884109
slope = 0.04987813304
intercept = 0.01001112826
r2 = 0.9999996643
  by: Km and Vmax from a Lineweaver-Burk plot  (least squares 1/v = a + b/S; Vmax = 1/a; Km = b/a)
  assumes: at least 3 points; S, v > 0; positive slope and intercept
```

In the worked example, S = [1, 2, 5, 10, 20, 50], v = [16.7, 28.6, 50, 66.7, 80, 90.9] go through the equation above and give Km = 4.982268904, Vmax = 99.88884109, slope = 0.04987813304, intercept = 0.01001112826. It applies only when at least 3 points; S, v > 0; positive slope and intercept: outside that it says what is wrong instead of returning a number.

Michaelis-Menten reaction rate — `cmb_michaelis_menten_rate`

The initial rate of an enzyme-catalysed reaction at substrate concentration S from Vmax and Km, and the fraction of Vmax.

$$v = \frac{V_{max}[S]}{K_m + [S]}$$

symbol		meaning	SI unit
`Vmax`	in	maximum rate	mol/(m^3*s)
`Km`	in	Michaelis constant	mol/m^3
`S`	in	substrate concentration	mol/m^3
`v`	out	initial rate	mol/(m^3*s)

symbol		meaning	SI unit
`v_uM_min`	out	rate in uM/min	—
`fraction`	out	v / Vmax	—

Applies when: Vmax, Km > 0; S >= 0; initial-rate conditions, S >> E.

```
$ cell-molecular-biology solve cmb_michaelis_menten_rate "Vmax=100 (umol/L)/min" "Km=50 (umol/L)"
"S=100 (umol/L)"
v = 0.001111111111 mol/(m^3*s)
v_uM_min = 66.66666667
fraction = 0.6666666667
  by: Michaelis-Menten reaction rate  (v = Vmax S / (Km + S))
  Vmax = 0.001666666667 mol/(m^3*s)
  Km = 0.05 mol/m^3
  S = 0.1 mol/m^3
  assumes: Vmax, Km > 0; S >= 0; initial-rate conditions, S >> E
```

In the worked example, Vmax = 100 (umol/L)/min, Km = 50 (umol/L), S = 100 (umol/L) go through the equation above and give v = 0.001111111111 mol/(m^3*s), v_uM_min = 66.66666667, fraction = 0.6666666667. It applies only when Vmax, Km > 0; S >= 0; initial-rate conditions, S >> E: outside that it says what is wrong instead of returning a number.

Mixed inhibition rate — `cmb_mixed_inhibition`

The rate, apparent Km and apparent Vmax of an enzyme with a mixed inhibitor that binds free enzyme (Ki) and the enzyme-substrate complex (Kii) with different affinities.

$$v = \frac{V_{max}[S]}{K_m\left(1+\frac{[I]}{K_i}\right)+[S]\left(1+\frac{[I]}{K_i'}\right)}$$

symbol		meaning	SI unit
`Vmax`	in	maximum rate without inhibitor	mol/(m^3*s)
`Km`	in	Michaelis constant	mol/m^3
`S`	in	substrate concentration	mol/m^3
`I`	in	inhibitor concentration	mol/m^3
`Ki`	in	dissociation constant of EI	mol/m^3
`Kii`	in	dissociation constant of ESI	mol/m^3
`v`	out	rate with the inhibitor	mol/(m^3*s)
`Km_app`	out	apparent Km	mol/m^3
`Vmax_app`	out	apparent Vmax	mol/(m^3*s)
`alpha`	out	Kii / Ki	—

Applies when: Vmax, Km, Ki, Kii > 0; S, I >= 0; rapid equilibrium.

```
$ cell-molecular-biology solve cmb_mixed_inhibition "Vmax=6 (umol/L)/min" "Km=50 (umol/L)" "S=100
(umol/L)" "I=20 (umol/L)" "Ki=10 (umol/L)" "Kii=40 (umol/L)"
v = 3.333333333e-05 mol/(m^3*s)
Km_app = 0.1 mol/m^3
Vmax_app = 6.666666667e-05 mol/(m^3*s)
alpha = 4
  by: Mixed inhibition rate  (v = Vmax S / (Km (1 + I/Ki) + S (1 + I/Kii)); Km_app = Km (1 +
I/Ki)/(1 + I/Kii); Vmax_app = Vmax/(1 + I/Kii))
  Vmax = 0.0001 mol/(m^3*s)
  Km = 0.05 mol/m^3
  S = 0.1 mol/m^3
  I = 0.02 mol/m^3
  Ki = 0.01 mol/m^3
  Kii = 0.04 mol/m^3
  assumes: Vmax, Km, Ki, Kii > 0; S, I >= 0; rapid equilibrium
```

In the worked example, Vmax = 6 (umol/L)/min, Km = 50 (umol/L), S = 100 (umol/L), I = 20 (umol/L), Ki = 10 (umol/L), Kii = 40 (umol/L) go through the equation above and give v = 3.333333333e-05 mol/(m^3*s), Km_app = 0.1 mol/m^3, Vmax_app = 6.666666667e-05 mol/(m^3*s), alpha = 4. It applies only when Vmax, Km, Ki, Kii > 0; S, I >= 0; rapid equilibrium: outside that it says what is wrong instead of returning a number.

Noncompetitive inhibition rate — `cmb_noncompetitive_inhibition`

The rate of an enzyme with a pure noncompetitive inhibitor (equal affinity for E and ES): Vmax falls by (1 + I/Ki), Km unchanged.

$$v = \frac{V_{max}[S]}{(K_m + [S])(1 + [I]/K_i)}$$

symbol		meaning	SI unit
`Vmax`	in	maximum rate without inhibitor	mol/(m^3*s)
`Km`	in	Michaelis constant	mol/m^3
`S`	in	substrate concentration	mol/m^3
`I`	in	inhibitor concentration	mol/m^3
`Ki`	in	inhibition (dissociation) constant	mol/m^3
`v`	out	rate with the inhibitor	mol/(m^3*s)
`Km_app`	out	apparent Km	mol/m^3
`Vmax_app`	out	apparent Vmax	mol/(m^3*s)
`percent_inhibition`	out	per cent inhibition at this S	—

Applies when: Vmax, Km, Ki > 0; S, I >= 0; rapid-equilibrium inhibitor binding.

```
$ cell-molecular-biology solve cmb_noncompetitive_inhibition "Vmax=6 (umol/L)/min" "Km=50 (umol/L)"
"S=100 (umol/L)" "I=20 (umol/L)" "Ki=10 (umol/L)"
v = 2.222222222e-05 mol/(m^3*s)
Km_app = 0.05 mol/m^3
Vmax_app = 3.333333333e-05 mol/(m^3*s)
percent_inhibition = 66.66666667
  by: Noncompetitive inhibition rate  (v = Vmax S / ((Km + S)(1 + I/Ki)))
  Vmax = 0.0001 mol/(m^3*s)
  Km = 0.05 mol/m^3
  S = 0.1 mol/m^3
  I = 0.02 mol/m^3
  Ki = 0.01 mol/m^3
  assumes: Vmax, Km, Ki > 0; S, I >= 0; rapid-equilibrium inhibitor binding
```

In the worked example, Vmax = 6 (umol/L)/min, Km = 50 (umol/L), S = 100 (umol/L), I = 20 (umol/L), Ki = 10 (umol/L) go through the equation above and give v = 2.222222222e-05 mol/(m^3*s), Km_app = 0.05 mol/m^3, Vmax_app = 3.333333333e-05 mol/(m^3*s), percent_inhibition = 66.66666667. It applies only when Vmax, Km, Ki > 0; S, I >= 0; rapid-equilibrium inhibitor binding: outside that it says what is wrong instead of returning a number.

Rate at another temperature from Q10 — `cmb_q10_temperature`

A biological rate at temperature T2 from its value at T1 and the temperature coefficient Q10, and the equivalent Arrhenius activation energy over that interval.

$$k_2 = k_1 Q_{10}^{(T_2 - T_1)/10}, \quad E_a = \frac{R \ln(k_2/k_1)}{1/T_1 - 1/T_2}$$

symbol		meaning	SI unit
`k1`	in	rate at T1 (any unit)	—
`T1`	in	first temperature (degC is converted)	K
`T2`	in	second temperature (degC is converted)	K
`Q10`	in	temperature coefficient Q10	—
`k2`	out	rate at T2 (unit of k1)	—
`ratio`	out	k2 / k1	—
`Ea`	out	equivalent activation energy	J/mol
`Ea_kJ_mol`	out	activation energy in kJ/mol	—

Applies when: k1, Q10 > 0; T1 != T2, both > 0 K.

```
$ cell-molecular-biology solve cmb_q10_temperature k1=10 "T1=298.15 K" "T2=308.15 K" Q10=2
k2 = 20
ratio = 2
Ea = 52948.86217 J/mol
Ea_kJ_mol = 52.94886217
  by: Rate at another temperature from Q10  (k2 = k1 Q10^((T2 - T1)/10); Ea = R ln(k2/k1) / (1/T1 -
1/T2))
  k1 = 10
  T1 = 298.15 K
  T2 = 308.15 K
  Q10 = 2
  assumes: k1, Q10 > 0; T1 != T2, both > 0 K
```

In the worked example, k1 = 10, T1 = 298.15 K, T2 = 308.15 K, Q10 = 2 go through the equation above and give k2 = 20, ratio = 2, Ea = 52948.86217 J/mol, Ea_kJ_mol = 52.94886217. It applies only when k1, Q10 > 0; T1 != T2, both > 0 K: outside that it says what is wrong instead of returning a number.

Substrate concentration for a fraction of Vmax — `cmb_substrate_for_fraction_of_vmax`

The substrate concentration at which a Michaelis-Menten enzyme runs at a fraction f of Vmax (e.g. 90 % needs 9 Km).

$$[S] = K_m \frac{f}{1 - f}$$

symbol		meaning	SI unit
`Km`	in	Michaelis constant	mol/m^3
`f`	in	fraction of Vmax (0 to 1)	—
`S`	out	required substrate concentration	mol/m^3
`S_uM`	out	S in uM	—
`S_over_Km`	out	S as a multiple of Km	—

Applies when: Km > 0; 0 < f < 1.

```
$ cell-molecular-biology solve cmb_substrate_for_fraction_of_vmax "Km=50 (umol/L)" f=0.9
S = 0.45 mol/m^3
S_uM = 450
S_over_Km = 9
  by: Substrate concentration for a fraction of Vmax  (S = Km f / (1 - f))
  Km = 0.05 mol/m^3
  f = 0.9
  assumes: Km > 0; 0 < f < 1
```

In the worked example, Km = 50 (umol/L), f = 0.9 go through the equation above and give S = 0.45 mol/m^3, S_uM = 450, S_over_Km = 9. It applies only when Km > 0; 0 < f < 1: outside that it says what is wrong instead of returning a number.

Turnover number kcat and catalytic efficiency kcat/Km — `cmb_kcat_catalytic_efficiency`

The turnover number kcat = Vmax/[E]t, the catalytic efficiency kcat/Km (compare with the diffusion limit of about 1e8-1e9 per M per s) and the time per turnover.

$$k_{cat} = \frac{V_{max}}{[E]_t}, \quad \eta = \frac{k_{cat}}{K_m}$$

symbol		meaning	SI unit
`Vmax`	in	maximum rate	mol/(m^3*s)
`Et`	in	total enzyme concentration	mol/m^3
`Km`	in	Michaelis constant	mol/m^3
`kcat`	out	turnover number	1/s
`kcat_per_s`	out	kcat in 1/s	—
`efficiency`	out	catalytic efficiency kcat/Km	m^3/(mol*s)
`efficiency_per_M_s`	out	kcat/Km in 1/(M s)	—
`turnover_time`	out	time per catalytic event 1/kcat	s

Applies when: Vmax, Et, Km > 0.

```
$ cell-molecular-biology solve cmb_kcat_catalytic_efficiency "Vmax=60 (umol/L)/min" "Et=10
(nmol/L)" "Km=25 (umol/L)"
kcat = 100 1/s
kcat_per_s = 100
efficiency = 4000 m^3/(mol*s)
efficiency_per_M_s = 4000000
turnover_time = 0.01 s
  by: Turnover number kcat and catalytic efficiency kcat/Km  (kcat = Vmax / [E]t; efficiency = kcat
/ Km)
  Vmax = 0.001 mol/(m^3*s)
  Et = 1e-05 mol/m^3
  Km = 0.025 mol/m^3
  assumes: Vmax, Et, Km > 0
```

In the worked example, Vmax = 60 (umol/L)/min, Et = 10 (nmol/L), Km = 25 (umol/L) go through the equation above and give kcat = 100 1/s, kcat_per_s = 100, efficiency = 4000 m^3/(mol*s), efficiency_per_M_s = 4000000. It applies only when Vmax, Et, Km > 0: outside that it says what is wrong instead of returning a number.

Uncompetitive inhibition rate — `cmb_uncompetitive_inhibition`

The rate of an enzyme with an uncompetitive inhibitor (binding the enzyme-substrate complex only): Km and Vmax both fall by (1 + I/Ki).

$$v = \frac{V_{max}[S]}{K_m + [S](1 + [I]/K_i)}$$

symbol		meaning	SI unit
`Vmax`	in	maximum rate without inhibitor	mol/(m^3*s)

symbol		meaning	SI unit
`Km`	in	Michaelis constant	mol/m^3
`S`	in	substrate concentration	mol/m^3
`I`	in	inhibitor concentration	mol/m^3
`Ki`	in	inhibition (dissociation) constant	mol/m^3
`v`	out	rate with the inhibitor	mol/(m^3*s)
`Km_app`	out	apparent Km	mol/m^3
`Vmax_app`	out	apparent Vmax	mol/(m^3*s)
`percent_inhibition`	out	per cent inhibition at this S	—

Applies when: Vmax, Km, Ki > 0; S, I >= 0; rapid-equilibrium inhibitor binding.

```
$ cell-molecular-biology solve cmb_uncompetitive_inhibition "Vmax=6 (umol/L)/min" "Km=50 (umol/L)"
"S=100 (umol/L)" "I=20 (umol/L)" "Ki=10 (umol/L)"
v = 2.857142857e-05 mol/(m^3*s)
Km_app = 0.01666666667 mol/m^3
Vmax_app = 3.333333333e-05 mol/(m^3*s)
percent_inhibition = 57.14285714
  by: Uncompetitive inhibition rate  (v = Vmax S / (Km + S (1 + I/Ki)))
  Vmax = 0.0001 mol/(m^3*s)
  Km = 0.05 mol/m^3
  S = 0.1 mol/m^3
  I = 0.02 mol/m^3
  Ki = 0.01 mol/m^3
  assumes: Vmax, Km, Ki > 0; S, I >= 0; rapid-equilibrium inhibitor binding
```

In the worked example, Vmax = 6 (umol/L)/min, Km = 50 (umol/L), S = 100 (umol/L), I = 20 (umol/L), Ki = 10 (umol/L) go through the equation above and give v = 2.857142857e-05 mol/(m^3*s), Km_app = 0.01666666667 mol/m^3, Vmax_app = 3.333333333e-05 mol/(m^3*s), percent_inhibition = 57.14285714. It applies only when Vmax, Km, Ki > 0; S, I >= 0; rapid-equilibrium inhibitor binding: outside that it says what is wrong instead of returning a number.

Enzyme kinetics and binding (10)

Complex concentration with ligand depletion (quadratic binding) — `cmb_ligand_depletion_binding`

The concentration of receptor-ligand complex when neither component is in excess (exact quadratic solution), with the fractions bound and the free ligand.

$$[RL] = \frac{(R_t + L_t + K_d) - \sqrt{(R_t + L_t + K_d)^2 - 4R_tL_t}}{2}$$

symbol		meaning	SI unit
`Rt`	in	total receptor concentration	mol/m^3
`Lt`	in	total ligand concentration	mol/m^3
`Kd`	in	dissociation constant	mol/m^3
`RL`	out	complex concentration	mol/m^3

symbol		meaning	SI unit
`fraction_receptor_bound`	out	RL / Rt	—
`free_ligand`	out	free ligand Lt - RL	mol/m^3
`fraction_ligand_bound`	out	RL / Lt	—

Applies when: Rt, Lt, Kd > 0; single site, 1:1 binding at equilibrium.

```
$ cell-molecular-biology solve cmb_ligand_depletion_binding "Rt=10 (nmol/L)" "Lt=20 (nmol/L)"
"Kd=10 (nmol/L)"
RL = 5.857864376e-06 mol/m^3
fraction_receptor_bound = 0.5857864376
free_ligand = 1.414213562e-05 mol/m^3
fraction_ligand_bound = 0.2928932188
  by: Complex concentration with ligand depletion (quadratic binding)  (RL = [(Rt + Lt + Kd) -
sqrt((Rt + Lt + Kd)^2 - 4 Rt Lt)] / 2)
  Rt = 1e-05 mol/m^3
  Lt = 2e-05 mol/m^3
  Kd = 1e-05 mol/m^3
  assumes: Rt, Lt, Kd > 0; single site, 1:1 binding at equilibrium
```

In the worked example, Rt = 10 (nmol/L), Lt = 20 (nmol/L), Kd = 10 (nmol/L) go through the equation above and give RL = 5.857864376e-06 mol/m^3, fraction_receptor_bound = 0.5857864376, free_ligand = 1.414213562e-05 mol/m^3, fraction_ligand_bound = 0.2928932188. It applies only when Rt, Lt, Kd > 0; single site, 1:1 binding at equilibrium: outside that it says what is wrong instead of returning a number.

Dissociation constant from association and dissociation rates — `cmb_kd_from_rates`

The equilibrium dissociation constant Kd = koff/kon, the association constant, the complex half-life and the residence time.

$$K_d = \frac{k_{off}}{k_{on}}, \quad t_{1/2} = \frac{\ln 2}{k_{off}}$$

symbol		meaning	SI unit
`kon`	in	association rate constant	m^3/(mol*s)
`koff`	in	dissociation rate constant	1/s
`Kd`	out	dissociation constant	mol/m^3
`Kd_nM`	out	Kd in nM	—
`Ka_per_M`	out	association constant in 1/M	—
`half_life`	out	complex half-life ln2/koff	s
`residence_time`	out	mean residence time 1/koff	s

Applies when: kon, koff > 0; simple 1:1 binding.

```
$ cell-molecular-biology solve cmb_kd_from_rates "kon=1000000 1/((mol/L)*s)" "koff=0.001 1/s"
Kd = 1e-06 mol/m^3
Kd_nM = 1
Ka_per_M = 1000
half_life = 693.1471806 s
residence_time = 1000 s
  by: Dissociation constant from association and dissociation rates  (Kd = koff/kon; t1/2 = ln
2/koff; tau = 1/koff)
  kon = 1000 m^3/(mol*s)
  koff = 0.001 1/s
  assumes: kon, koff > 0; simple 1:1 binding
```

In the worked example, kon = 1000000 1/((mol/L)*s), koff = 0.001 1/s go through the equation above and give Kd = 1e-06 mol/m^3, Kd_nM = 1, Ka_per_M = 1000, half_life = 693.1471806 s. It applies only when kon, koff > 0; simple 1:1 binding: outside that it says what is wrong instead of returning a number.

Fraction of receptor bound at a free ligand concentration — `cmb_fraction_bound`

The fractional occupancy L/(Kd + L) of a single-site receptor at free ligand concentration L, and the bound concentration for a receptor total Rt (ligand not depleted).

$$\theta = \frac{[L]}{K_d + [L]}$$

symbol		meaning	SI unit
`L`	in	free ligand concentration	mol/m^3
`Kd`	in	dissociation constant	mol/m^3
`Rt`	in	total receptor concentration (optional) (optional)	mol/m^3
`theta`	out	fraction bound	—
`theta_percent`	out	per cent bound	—
`bound`	out	bound complex concentration (when Rt is given)	mol/m^3

Applies when: L, Kd > 0; single independent site; ligand in excess (free = total).

```
$ cell-molecular-biology solve cmb_fraction_bound "L=10 (nmol/L)" "Kd=5 (nmol/L)"
theta = 0.6666666667
theta_percent = 66.66666667
  by: Fraction of receptor bound at a free ligand concentration  (theta = L / (Kd + L))
  L = 1e-05 mol/m^3
  Kd = 5e-06 mol/m^3
  assumes: L, Kd > 0; single independent site; ligand in excess (free = total)
```

In the worked example, L = 10 (nmol/L), Kd = 5 (nmol/L) go through the equation above and give theta = 0.6666666667, theta_percent = 66.66666667. It applies only when L, Kd > 0; single independent site; ligand in excess (free = total): outside that it says what is wrong instead of returning a number.

Fractional saturation by the Hill equation — `cmb_hill_equation`

The fractional saturation (or response) of a cooperative binding protein at ligand concentration L, with half-saturation K and Hill coefficient n.

$$\theta = \frac{[L]^n}{K_{0.5}^n + [L]^n}$$

symbol		meaning	SI unit
`L`	in	free ligand concentration	mol/m^3
`K`	in	ligand concentration for half saturation (K0.5)	mol/m^3
`n`	in	Hill coefficient	—
`theta`	out	fractional saturation	—
`theta_percent`	out	saturation in per cent	—

Applies when: L, K, n > 0.

```
$ cell-molecular-biology solve cmb_hill_equation "L=26 (umol/L)" "K=26 (umol/L)" n=2.8
theta = 0.5
theta_percent = 50
  by: Fractional saturation by the Hill equation  (theta = L^n / (K^n + L^n))
  L = 0.026 mol/m^3
  K = 0.026 mol/m^3
  n = 2.8
  assumes: L, K, n > 0
```

In the worked example, L = 26 (umol/L), K = 26 (umol/L), n = 2.8 go through the equation above and give theta = 0.5, theta_percent = 50. It applies only when L, K, n > 0: outside that it says what is wrong instead of returning a number.

Fractional saturation in the MWC (concerted) model — `cmb_mwc_saturation`

The fractional saturation and the fraction of molecules in the R state of an allosteric protein with n sites in the Monod-Wyman-Changeux model (L0 = [T]/[R] without ligand, c = KR/KT).

$$Y = \frac{\alpha(1+\alpha)^{n-1} + L_0 c\alpha(1+c\alpha)^{n-1}}{(1+\alpha)^n + L_0(1+c\alpha)^n}$$

symbol		meaning	SI unit
`S`	in	ligand concentration	mol/m^3
`KR`	in	dissociation constant of a site in the R state	mol/m^3
`c`	in	ratio KR/KT (0 < c < 1)	—
`L0`	in	allosteric constant [T0]/[R0]	—
`n`	in	number of binding sites	—
`Y`	out	fractional saturation	—
`R_fraction`	out	fraction of molecules in the R state	—

Applies when: S, KR, c, L0 > 0; 1 <= n <= 12; concerted R/T transition.

```
$ cell-molecular-biology solve cmb_mwc_saturation "S=10 (umol/L)" "KR=2 (umol/L)" c=0.01 L0=9000
n=4
Y = 0.1308425393
R_fraction = 0.1059208076
  by: Fractional saturation in the MWC (concerted) model  (alpha = S/KR; Y = [alpha (1 + alpha)^(n-
1) + L0 c alpha (1 + c alpha)^(n-1)] / [(1 + alpha)^n + L0 (1 + c alpha)^n])
  S = 0.01 mol/m^3
  KR = 0.002 mol/m^3
  c = 0.01
  L0 = 9000
  assumes: S, KR, c, L0 > 0; 1 <= n <= 12; concerted R/T transition
```

In the worked example, S = 10 (umol/L), KR = 2 (umol/L), c = 0.01, L0 = 9000, n = 4 go through the equation above and give Y = 0.1308425393, R_fraction = 0.1059208076. It applies only when S, KR, c, L0 > 0; 1 <= n <= 12; concerted R/T transition: outside that it says what is wrong instead of returning a number.

Hill coefficient from EC10 and EC90 — `cmb_hill_coefficient_from_ec`

The Hill coefficient n = ln 81 / ln(EC90/EC10) and the EC50 (their geometric mean) of a Hill dose-response curve (81-fold range for n = 1).

$$n_H = \frac{\ln 81}{\ln(EC_{90}/EC_{10})}$$

symbol		meaning	SI unit
`EC10`	in	concentration giving 10 % of the maximum	mol/m^3
`EC90`	in	concentration giving 90 % of the maximum	mol/m^3
`n`	out	Hill coefficient	—
`EC50`	out	half-maximal concentration	mol/m^3
`ratio`	out	EC90 / EC10	—

Applies when: 0 < EC10 < EC90; a Hill curve.

```
$ cell-molecular-biology solve cmb_hill_coefficient_from_ec "EC10=1 (umol/L)" "EC90=81 (umol/L)"
n = 1
EC50 = 0.009 mol/m^3
ratio = 81
  by: Hill coefficient from EC10 and EC90  (n = ln 81 / ln(EC90/EC10); EC50 = sqrt(EC10 EC90))
  EC10 = 0.001 mol/m^3
  EC90 = 0.081 mol/m^3
  assumes: 0 < EC10 < EC90; a Hill curve
```

In the worked example, EC10 = 1 (umol/L), EC90 = 81 (umol/L) go through the equation above and give n = 1, EC50 = 0.009 mol/m^3, ratio = 81. It applies only when 0 < EC10 < EC90; a Hill curve: outside that it says what is wrong instead of returning a number.

Kd and Bmax from a Scatchard plot — `cmb_scatchard_fit`

The dissociation constant and the total binding sites from saturation-binding data by the Scatchard least-squares line B/F = Bmax/Kd - B/Kd.

$$\frac{B}{F} = \frac{B_{max}}{K_d} - \frac{B}{K_d}$$

symbol		meaning	SI unit
`B`	in	bound ligand concentrations (any one unit, e.g. nM or fmol/mg)	—
`F`	in	free ligand concentrations (e.g. nM)	—
`Kd`	out	dissociation constant (unit of F)	—
`Bmax`	out	total binding sites (unit of B)	—
`slope`	out	slope -1/Kd	—
`intercept`	out	intercept Bmax/Kd	—
`r2`	out	r^2 of the Scatchard line	—

Applies when: at least 3 points; B, F > 0; negative slope (one class of sites).

```
$ cell-molecular-biology solve cmb_scatchard_fit "B=[16.7, 28.6, 50, 66.7, 80]" "F=[1, 2, 5, 10,
20]"
Kd = 4.986488927
Bmax = 99.93037658
slope = -0.2005419073
intercept = 20.04022831
r2 = 0.9999971119
  by: Kd and Bmax from a Scatchard plot  (least squares B/F = a + m B; Kd = -1/m; Bmax = -a/m)
  assumes: at least 3 points; B, F > 0; negative slope (one class of sites)
```

In the worked example, B = [16.7, 28.6, 50, 66.7, 80], F = [1, 2, 5, 10, 20] go through the equation above and give Kd = 4.986488927, Bmax = 99.93037658, slope = -0.2005419073, intercept = 20.04022831. It applies only when at least 3 points; B, F > 0; negative slope (one class of sites): outside that it says what is wrong instead of returning a number.

Ligand concentration for a given saturation (Hill) — `cmb_hill_ligand_for_fraction`

The ligand concentration that gives a fractional saturation theta for a cooperative protein with half-saturation K and Hill coefficient n.

$$[L] = K_{0.5}\left(\frac{\theta}{1-\theta}\right)^{1/n}$$

symbol		meaning	SI unit
`K`	in	half-saturation concentration K0.5	mol/m^3
`n`	in	Hill coefficient	—
`theta`	in	fractional saturation (0 to 1)	—
`L`	out	required ligand concentration	mol/m^3
`L_uM`	out	L in uM	—

Applies when: K, n > 0; 0 < theta < 1.

```
$ cell-molecular-biology solve cmb_hill_ligand_for_fraction "K=26 (umol/L)" n=2.8 theta=0.9
L = 0.05698679654 mol/m^3
L_uM = 56.98679654
  by: Ligand concentration for a given saturation (Hill)  (L = K (theta/(1 - theta))^(1/n))
  K = 0.026 mol/m^3
  n = 2.8
  theta = 0.9
  assumes: K, n > 0; 0 < theta < 1
```

In the worked example, K = 26 (umol/L), n = 2.8, theta = 0.9 go through the equation above and give L = 0.05698679654 mol/m^3, L_uM = 56.98679654. It applies only when K, n > 0; 0 < theta < 1: outside that it says what is wrong instead of returning a number.

Receptor occupancy with a competing ligand — `cmb_competitive_occupancy`

The fractions of a receptor occupied by a ligand and by a competitor binding the same site, the free fraction and the apparent Kd of the ligand in the competitor's presence (no depletion).

$$\theta_L = \frac{[L]/K_d}{1 + [L]/K_d + [I]/K_i}$$

symbol		meaning	SI unit
`L`	in	free ligand concentration	mol/m^3
`Kd`	in	ligand dissociation constant	mol/m^3
`I`	in	free competitor concentration	mol/m^3
`Ki`	in	competitor dissociation constant	mol/m^3
`occupancy_L`	out	fraction occupied by the ligand	—
`occupancy_I`	out	fraction occupied by the competitor	—
`free_fraction`	out	fraction of free receptor	—
`Kd_app`	out	apparent Kd of the ligand Kd (1 + I/Ki)	mol/m^3

Applies when: Kd, Ki > 0; L, I >= 0; one site, mutually exclusive binding, no depletion.

```
$ cell-molecular-biology solve cmb_competitive_occupancy "L=10 (nmol/L)" "Kd=5 (nmol/L)" "I=20
(nmol/L)" "Ki=10 (nmol/L)"
occupancy_L = 0.4
occupancy_I = 0.4
free_fraction = 0.2
Kd_app = 1.5e-05 mol/m^3
  by: Receptor occupancy with a competing ligand  (occupancy_L = (L/Kd) / (1 + L/Kd + I/Ki))
  L = 1e-05 mol/m^3
  Kd = 5e-06 mol/m^3
  I = 2e-05 mol/m^3
  Ki = 1e-05 mol/m^3
  assumes: Kd, Ki > 0; L, I >= 0; one site, mutually exclusive binding, no depletion
```

In the worked example, L = 10 (nmol/L), Kd = 5 (nmol/L), I = 20 (nmol/L), Ki = 10 (nmol/L) go through the equation above and give occupancy_L = 0.4, occupancy_I = 0.4, free_fraction = 0.2, Kd_app = 1.5e-05

mol/m^3. It applies only when Kd, Ki > 0; L, I >= 0; one site, mutually exclusive binding, no depletion: outside that it says what is wrong instead of returning a number.

Standard binding free energy from Kd — `cmb_binding_free_energy`

The standard free energy of binding dG = RT ln(Kd / 1 M) (negative for favourable binding) at temperature T.

$$\Delta G^\circ = RT \ln \frac{K_d}{1\ \mathrm{M}}$$

symbol		meaning	SI unit
`Kd`	in	dissociation constant	mol/m^3
`T`	in	absolute temperature (degC is converted)	K
`dG`	out	standard binding free energy	J/mol
`dG_kJ_mol`	out	dG in kJ/mol	—
`dG_kcal_mol`	out	dG in kcal/mol	—

Applies when: Kd, T > 0; standard state 1 M.

```
$ cell-molecular-biology solve cmb_binding_free_energy "Kd=10 (nmol/L)" "T=298.15 K"
dG = -45664.07602 J/mol
dG_kJ_mol = -45.66407602
dG_kcal_mol = -10.9139761
  by: Standard binding free energy from Kd  (dG = R T ln(Kd/1 M) = -R T ln Ka)
  Kd = 1e-05 mol/m^3
  T = 298.15 K
  assumes: Kd, T > 0; standard state 1 M
```

In the worked example, Kd = 10 (nmol/L), T = 298.15 K go through the equation above and give dG = -45664.07602 J/mol, dG_kJ_mol = -45.66407602, dG_kcal_mol = -10.9139761. It applies only when Kd, T > 0; standard state 1 M: outside that it says what is wrong instead of returning a number.

Membranes and diffusion (14)

Cell volume after an osmotic change (Boyle-van 't Hoff) — `cmb_cell_volume_osmotic`

The equilibrium volume of a cell moved from osmolarity osm0 to osm1, with an osmotically inactive volume fraction b (an ideal osmometer).

$$V = V_b + (V_0 - V_b)\,\frac{\mathrm{osm}_0}{\mathrm{osm}_1}$$

symbol		meaning	SI unit
`V0`	in	initial cell volume	m^3
`b`	in	osmotically inactive fraction of the initial volume (0 to 1)	—
`osm0`	in	initial (isotonic) osmolarity	mol/m^3
`osm1`	in	new outside osmolarity	mol/m^3

symbol		meaning	SI unit
`V`	out	new equilibrium cell volume	m^3
`V_fL`	out	new volume in femtolitres	—
`ratio`	out	new volume relative to the initial volume	—

Applies when: V0, osm0, osm1 > 0; 0 <= b < 1; impermeant solutes, water in equilibrium.

```
$ cell-molecular-biology solve cmb_cell_volume_osmotic "V0=90 fL" b=0.4 "osm0=300 mmol/L" "osm1=600
mmol/L"
V = 6.3e-17 m^3
V_fL = 63
ratio = 0.7
  by: Cell volume after an osmotic change (Boyle-van 't Hoff)  (V = Vb + (V0 - Vb) osm0 / osm1 with
Vb = b V0)
  V0 = 9e-17 m^3
  b = 0.4
  osm0 = 300 mol/m^3
  osm1 = 600 mol/m^3
  assumes: V0, osm0, osm1 > 0; 0 <= b < 1; impermeant solutes, water in equilibrium
```

In the worked example, V0 = 90 fL, b = 0.4, osm0 = 300 mmol/L, osm1 = 600 mmol/L go through the equation above and give V = 6.3e-17 m^3, V_fL = 63, ratio = 0.7. It applies only when V0, osm0, osm1 > 0; 0 <= b < 1; impermeant solutes, water in equilibrium: outside that it says what is wrong instead of returning a number.

Diffusion coefficient from a FRAP half-time (Soumpasis) — `cmb_frap_diffusion`

The two-dimensional diffusion coefficient from the half-time of fluorescence recovery after photobleaching a uniform circular spot of radius w (Soumpasis' exact solution, D = 0.224 w^2 / t_half).

$$f(t) = e^{-2\tau_D/t}\left[I_0\left(\tfrac{2\tau_D}{t}\right) + I_1\left(\tfrac{2\tau_D}{t}\right)\right], \quad D = \frac{w^2}{4\tau_D}$$

symbol		meaning	SI unit
`w`	in	radius of the bleached spot	m
`t_half`	in	half-time of recovery	s
`D`	out	diffusion coefficient	m^2/s
`D_um2_s`	out	D in um^2/s	—
`tauD`	out	characteristic diffusion time w^2/(4D)	s
`gamma`	out	tauD / t_half (about 1.12)	—

Applies when: w, t_half > 0; 2-D free diffusion, uniform circular bleach, full mobile fraction.

```
$ cell-molecular-biology solve cmb_frap_diffusion "w=1 um" "t_half=0.5 s"
D = 4.472955755e-13 m^2/s
D_um2_s = 0.4472955755
tauD = 0.5589145382 s
gamma = 1.117829076
  by: Diffusion coefficient from a FRAP half-time (Soumpasis)  (f(t) = exp(-2 tauD/t)[I0(2 tauD/t)
+ I1(2 tauD/t)]; solve f(t_half) = 1/2 for tauD; D = w^2/(4 tauD))
  w = 1e-06 m
  t_half = 0.5 s
  assumes: w, t_half > 0; 2-D free diffusion, uniform circular bleach, full mobile fraction
```

In the worked example, w = 1 um, t_half = 0.5 s go through the equation above and give D = 4.472955755e-13 m^2/s, D_um2_s = 0.4472955755, tauD = 0.5589145382 s, gamma = 1.117829076. It applies only when w, t_half > 0; 2-D free diffusion, uniform circular bleach, full mobile fraction: outside that it says what is wrong instead of returning a number.

Diffusion-limited association rate constant (Smoluchowski) — `cmb_diffusion_limited_rate`

The maximum (diffusion-limited) bimolecular association rate constant of two spheres that react on contact.

$$k_D = 4\pi(D_A + D_B)(r_A + r_B)N_A$$

symbol		meaning	SI unit
`DA`	in	diffusion coefficient of molecule A	m^2/s
`DB`	in	diffusion coefficient of molecule B	m^2/s
`rA`	in	radius of molecule A	m
`rB`	in	radius of molecule B	m
`k`	out	diffusion-limited rate constant	m^3/(mol*s)
`k_per_M_s`	out	rate constant in 1/(M s)	—

Applies when: all inputs > 0; reaction on every contact, no interaction forces.

```
$ cell-molecular-biology solve cmb_diffusion_limited_rate "DA=100 um^2/s" "DB=1000 um^2/s" "rA=2
nm" "rB=0.5 nm"
k = 20811024.49 m^3/(mol*s)
k_per_M_s = 2.081102449e+10
  by: Diffusion-limited association rate constant (Smoluchowski)  (k = 4 pi (DA + DB)(rA + rB) N_A)
  DA = 1e-10 m^2/s
  DB = 1e-09 m^2/s
  rA = 2e-09 m
  rB = 5e-10 m
  assumes: all inputs > 0; reaction on every contact, no interaction forces
```

In the worked example, DA = 100 um^2/s, DB = 1000 um^2/s, rA = 2 nm, rB = 0.5 nm go through the equation above and give k = 20811024.49 m^3/(mol*s), k_per_M_s = 2.081102449e+10. It applies only when all inputs > 0; reaction on every contact, no interaction forces: outside that it says what is wrong instead of returning a number.

Donnan equilibrium with an impermeant anion — `cmb_donnan_equilibrium`

The inside cation and anion concentrations, the Donnan ratio, the Donnan potential and the osmotic excess when a cell holds impermeant anions (charge -zA) bathed in a monovalent salt (KCl) of concentration c.

$$[\mathrm{Cl}]_i = \frac{-z_A A + \sqrt{z_A^2 A^2 + 4c^2}}{2}, \quad E = \frac{RT}{F} \ln \frac{c}{[\mathrm{K}]_i}$$

symbol		meaning	SI unit
`A`	in	concentration of impermeant anions inside	mol/m^3
`zA`	in	magnitude of the impermeant anion's charge (1, 2 or 3)	—
`c`	in	outside concentration of the monovalent salt	mol/m^3
`T`	in	absolute temperature (degC is converted)	K
`K_in`	out	inside cation concentration	mol/m^3
`Cl_in`	out	inside permeant anion concentration	mol/m^3
`ratio`	out	Donnan ratio K_in / K_out = Cl_out / Cl_in	—
`E`	out	Donnan potential (inside minus outside)	V
`E_mV`	out	Donnan potential in mV	—
`excess_mOsm`	out	inside minus outside osmolarity in mOsm/L	—

Applies when: A, c, T > 0; zA = 1, 2 or 3; ideal solutions, electroneutral compartments.

```
$ cell-molecular-biology solve cmb_donnan_equilibrium "A=100 (mmol/L)" zA=1 "c=100 (mmol/L)"
"T=298.15 K"
K_in = 161.8033989 mol/m^3
Cl_in = 61.80339887 mol/m^3
ratio = 1.618033989
E = -0.01236357289 V
E_mV = -12.36357289
excess_mOsm = 123.6067977
  by: Donnan equilibrium with an impermeant anion  (K_in Cl_in = c^2 and K_in = Cl_in + zA A: Cl_in
= (-zA A + sqrt(zA^2 A^2 + 4c^2))/2; E = (RT/F) ln(c/K_in))
  A = 100 mol/m^3
  c = 100 mol/m^3
  T = 298.15 K
  assumes: A, c, T > 0; zA = 1, 2 or 3; ideal solutions, electroneutral compartments
```

In the worked example, A = 100 (mmol/L), zA = 1, c = 100 (mmol/L), T = 298.15 K go through the equation above and give K_in = 161.8033989 mol/m^3, Cl_in = 61.80339887 mol/m^3, ratio = 1.618033989, E = -0.01236357289 V. It applies only when A, c, T > 0; zA = 1, 2 or 3; ideal solutions, electroneutral compartments: outside that it says what is wrong instead of returning a number.

Equilibration of a solute across a spherical cell's membrane — `cmb_cell_equilibration`

The internal concentration of a permeant solute at time t for a spherical cell (A/V = 3/r) bathed at a constant outside concentration, with the time constant and half-time.

$$C(t) = C_{out} + (C_0 - C_{out})\, e^{-3Pt/r}$$

symbol		meaning	SI unit
`r`	in	cell radius	m
`P`	in	membrane permeability coefficient	m/s
`C_out`	in	constant outside concentration	mol/m^3
`C0`	in	initial inside concentration	mol/m^3
`t`	in	elapsed time	s
`C`	out	inside concentration at time t	mol/m^3
`tau`	out	time constant r/(3P)	s
`t_half`	out	half-time of equilibration	s

Applies when: r, P > 0; C_out, C0, t >= 0; well-mixed cell of constant volume, outside unchanged.

```
$ cell-molecular-biology solve cmb_cell_equilibration "r=10 um" "P=1e-05 cm/s" "C_out=10 (mmol/L)"
"C0=0 (mmol/L)" "t=30 s"
C = 5.934303403 mol/m^3
tau = 33.33333333 s
t_half = 23.10490602 s
  by: Equilibration of a solute across a spherical cell's membrane  (dC/dt = P (A/V) (C_out - C),
A/V = 3/r: C = C_out + (C0 - C_out) exp(-3Pt/r))
  r = 1e-05 m
  P = 1e-07 m/s
  C_out = 10 mol/m^3
  C0 = 0 mol/m^3
  t = 30 s
  assumes: r, P > 0; C_out, C0, t >= 0; well-mixed cell of constant volume, outside unchanged
```

In the worked example, r = 10 um, P = 1e-05 cm/s, C_out = 10 (mmol/L), C0 = 0 (mmol/L), t = 30 s go through the equation above and give C = 5.934303403 mol/m^3, tau = 33.33333333 s, t_half = 23.10490602 s. It applies only when r, P > 0; C_out, C0, t >= 0; well-mixed cell of constant volume, outside unchanged: outside that it says what is wrong instead of returning a number.

Fick's first law: steady diffusion flux — `cmb_fick_flux`

The steady one-dimensional diffusive flux between two concentrations a distance apart (positive from side 1 to side 2).

$$J = -D\frac{dC}{dx} = D\,\frac{C_1 - C_2}{\Delta x}$$

symbol		meaning	SI unit
`D`	in	diffusion coefficient of the solute	m^2/s
`C1`	in	concentration on side 1	mol/m^3
`C2`	in	concentration on side 2	mol/m^3
`dx`	in	distance between the two sides	m
`J`	out	molar flux from side 1 to side 2	mol/(m^2*s)
`J_nmol_cm2_s`	out	flux in nmol per cm^2 per second	—

Applies when: D > 0, dx > 0; C1, C2 >= 0; steady state, linear profile.

```
$ cell-molecular-biology solve cmb_fick_flux "D=1000 um^2/s" "C1=10 (mmol/L)" "C2=2 (mmol/L)"
"dx=10 um"
J = 0.0008 mol/(m^2*s)
J_nmol_cm2_s = 80
  by: Fick's first law: steady diffusion flux  (J = D (C1 - C2) / dx (Fick's first law with a
linear steady profile))
  D = 1e-09 m^2/s
  C1 = 10 mol/m^3
  C2 = 2 mol/m^3
  dx = 1e-05 m
  assumes: D > 0, dx > 0; C1, C2 >= 0; steady state, linear profile
```

In the worked example, D = 1000 um^2/s, C1 = 10 (mmol/L), C2 = 2 (mmol/L), dx = 10 um go through the equation above and give J = 0.0008 mol/(m^2*s), J_nmol_cm2_s = 80. It applies only when D > 0, dx > 0; C1, C2 >= 0; steady state, linear profile: outside that it says what is wrong instead of returning a number.

Macroscopic current through ion channels — `cmb_channel_current`

The whole-cell current I = N Po gamma (V - E) through N channels of single-channel conductance gamma and open probability Po, and the single-channel current.

$$I = NP_o\gamma\left(V - E_{rev}\right)$$

symbol		meaning	SI unit
`N`	in	number of channels	—
`Po`	in	open probability (0 to 1)	—
`gamma`	in	single-channel conductance	S
`V`	in	membrane potential	V
`E`	in	reversal potential of the channel	V
`I`	out	macroscopic current (positive outward)	A
`I_pA`	out	current in pA	—
`G`	out	total open conductance N Po gamma	S
`i_single`	out	single open-channel current gamma (V - E)	A

Applies when: N >= 1; 0 <= Po <= 1; gamma > 0; ohmic open channels.

```
$ cell-molecular-biology solve cmb_channel_current N=1000 Po=0.3 "gamma=20 pS" "V=-40 mV" "E=-90
mV"
I = 3e-10 A
I_pA = 300
G = 6e-09 S
i_single = 1e-12 A
  by: Macroscopic current through ion channels  (I = N Po gamma (V - E))
  Po = 0.3
  gamma = 2e-11 S
  V = -0.04 V
  E = -0.09 V
  assumes: N >= 1; 0 <= Po <= 1; gamma > 0; ohmic open channels
```

In the worked example, N = 1000, Po = 0.3, gamma = 20 pS, V = -40 mV, E = -90 mV go through the equation above and give I = 3e-10 A, I_pA = 300, G = 6e-09 S, i_single = 1e-12 A. It applies only when N >= 1; 0 <= Po <= 1; gamma > 0; ohmic open channels: outside that it says what is wrong instead of returning a number.

Membrane permeability coefficient and flux — `cmb_membrane_permeability`

The permeability coefficient P = K D / L of a lipid membrane for a solute with partition coefficient K, and the inward flux P (C_out - C_in).

$$P = \frac{KD}{L}, \quad J = P\,(C_{out} - C_{in})$$

symbol		meaning	SI unit
`D`	in	diffusion coefficient of the solute inside the membrane	m^2/s
`K`	in	membrane/water partition coefficient	—
`L`	in	membrane thickness	m
`C_out`	in	concentration outside the cell	mol/m^3
`C_in`	in	concentration inside the cell	mol/m^3
`P`	out	permeability coefficient	m/s
`P_cm_s`	out	permeability coefficient in cm/s	—
`J`	out	inward flux (positive into the cell)	mol/(m^2*s)

Applies when: D, K, L > 0; concentrations >= 0; steady state, linear profile in the membrane.

```
$ cell-molecular-biology solve cmb_membrane_permeability "D=100 um^2/s" K=0.01 "L=5 nm" "C_out=10
(mmol/L)" "C_in=1 (mmol/L)"
P = 0.0002 m/s
P_cm_s = 0.02
J = 0.0018 mol/(m^2*s)
  by: Membrane permeability coefficient and flux  (P = K D / L; J = P (C_out - C_in))
  D = 1e-10 m^2/s
  K = 0.01
  L = 5e-09 m
  C_out = 10 mol/m^3
  C_in = 1 mol/m^3
  assumes: D, K, L > 0; concentrations >= 0; steady state, linear profile in the membrane
```

In the worked example, D = 100 um^2/s, K = 0.01, L = 5 nm, C_out = 10 (mmol/L), C_in = 1 (mmol/L) go through the equation above and give P = 0.0002 m/s, P_cm_s = 0.02, J = 0.0018 mol/(m^2*s). It applies only when D, K, L > 0; concentrations >= 0; steady state, linear profile in the membrane: outside that it says what is wrong instead of returning a number.

Nernst equilibrium potential of an ion across a cell membrane — `cmb_nernst_potential`

The equilibrium (reversal) potential inside relative to outside for an ion of charge z from its outside and inside concentrations.

$$E = \frac{RT}{zF} \ln \frac{C_{out}}{C_{in}}$$

symbol		meaning	SI unit
`z`	in	charge number of the ion (e.g. 1 for K+, 2 for Ca2+, -1 for Cl-)	—
`C_out`	in	outside concentration	mol/m^3
`C_in`	in	inside concentration	mol/m^3
`T`	in	absolute temperature (degC is converted)	K
`E`	out	equilibrium potential (inside minus outside)	V
`E_mV`	out	equilibrium potential in mV	—

Applies when: z a non-zero whole number; concentrations > 0; T > 0.

```
$ cell-molecular-biology solve cmb_nernst_potential z=1 "C_out=5 (mmol/L)" "C_in=140 (mmol/L)"
"T=310.15 K"
E = -0.08905869404 V
E_mV = -89.05869404
  by: Nernst equilibrium potential of an ion across a cell membrane  (E = (R T / z F) ln(C_out /
C_in))
  C_out = 5 mol/m^3
  C_in = 140 mol/m^3
  T = 310.15 K
  assumes: z a non-zero whole number; concentrations > 0; T > 0
```

In the worked example, z = 1, C_out = 5 (mmol/L), C_in = 140 (mmol/L), T = 310.15 K go through the equation above and give E = -0.08905869404 V, E_mV = -89.05869404. It applies only when z a non-zero whole number; concentrations > 0; T > 0: outside that it says what is wrong instead of returning a number.

Osmolarity of a solution and its tonicity for a cell — `cmb_osmolarity_tonicity`

The total osmolarity of a solution of several solutes (sum of i c) and whether it is hypertonic, hypotonic or isotonic (within 1 %) relative to the cell's osmolarity, assuming impermeant solutes.

$$\mathrm{osm} = \sum_k i_k c_k$$

symbol		meaning	SI unit
`c`	in	solute concentrations in mM (one per solute)	—

symbol		meaning	SI unit
`i`	in	van 't Hoff factor of each solute	—
`osm_cell`	in	osmolarity of the cell interior	mol/m^3
`osmolarity_mOsm`	out	osmolarity of the solution in mOsm/L	—
`difference_mOsm`	out	solution minus cell osmolarity in mOsm/L	—
`tonicity`	out	hypertonic, hypotonic or isotonic (within 1 %)	—

Applies when: one factor per concentration; solutes impermeant (tonicity = osmolarity).

```
$ cell-molecular-biology solve cmb_osmolarity_tonicity "c=[150]" "i=[2]" "osm_cell=300 mmol/L"
osmolarity_mOsm = 300
difference_mOsm = 0
tonicity = isotonic
  by: Osmolarity of a solution and its tonicity for a cell  (osmolarity = sum i_k c_k; compared
with the cell's osmolarity (isotonic within 1 %))
  osm_cell = 300 mol/m^3
  assumes: one factor per concentration; solutes impermeant (tonicity = osmolarity)
```

In the worked example, c = [150], i = [2], osm_cell = 300 mmol/L go through the equation above and give osmolarity_mOsm = 300, difference_mOsm = 0, tonicity = isotonic. It applies only when one factor per concentration; solutes impermeant (tonicity = osmolarity): outside that it says what is wrong instead of returning a number.

Osmotic pressure (van 't Hoff) — `cmb_osmotic_pressure`

The osmotic pressure of a dilute solution from the solute concentration, the van 't Hoff factor and the temperature.

$$\Pi = i\,c\,R\,T$$

symbol		meaning	SI unit
`i`	in	van 't Hoff factor (particles per formula unit, e.g. 2 for NaCl)	—
`c`	in	solute concentration	mol/m^3
`T`	in	absolute temperature (degC is converted)	K
`pi`	out	osmotic pressure	Pa
`pi_atm`	out	osmotic pressure in atm	—
`pi_kPa`	out	osmotic pressure in kPa	—

Applies when: i, c, T > 0; ideal dilute solution.

```
$ cell-molecular-biology solve cmb_osmotic_pressure i=2 "c=150 (mmol/L)" "T=310.15 K"
pi = 773619.1743 Pa
pi_atm = 7.635027627
pi_kPa = 773.6191743
  by: Osmotic pressure (van 't Hoff)  (Pi = i c R T)
  i = 2
  c = 150 mol/m^3
  T = 310.15 K
  assumes: i, c, T > 0; ideal dilute solution
```

In the worked example, i = 2, c = 150 (mmol/L), T = 310.15 K go through the equation above and give pi = 773619.1743 Pa, pi_atm = 7.635027627, pi_kPa = 773.6191743. It applies only when i, c, T > 0; ideal dilute solution: outside that it says what is wrong instead of returning a number.

Root-mean-square displacement of a diffusing particle — `cmb_rms_displacement`

The mean-square and root-mean-square displacement of a particle diffusing freely for a time t in 1, 2 or 3 dimensions.

$$\langle r^2 \rangle = 2dDt, \quad r_{\mathrm{rms}} = \sqrt{2dDt}$$

symbol		meaning	SI unit
`D`	in	diffusion coefficient	m^2/s
`t`	in	elapsed time	s
`dims`	in	number of dimensions (1, 2 or 3)	—
`rms`	out	root-mean-square displacement	m
`rms_um`	out	root-mean-square displacement in micrometres	—
`msd`	out	mean-square displacement	m^2

Applies when: D > 0, t > 0; dims = 1, 2 or 3.

```
$ cell-molecular-biology solve cmb_rms_displacement "D=1000 um^2/s" "t=1 s" dims=3
rms = 7.745966692e-05 m
rms_um = 77.45966692
msd = 6e-09 m^2
  by: Root-mean-square displacement of a diffusing particle  (<r^2> = 2 d D t; rms = sqrt(2 d D t))
  D = 1e-09 m^2/s
  t = 1 s
  assumes: D > 0, t > 0; dims = 1, 2 or 3
```

In the worked example, D = 1000 um^2/s, t = 1 s, dims = 3 go through the equation above and give rms = 7.745966692e-05 m, rms_um = 77.45966692, msd = 6e-09 m^2. It applies only when D > 0, t > 0; dims = 1, 2 or 3: outside that it says what is wrong instead of returning a number.

Stokes-Einstein diffusion coefficient — `cmb_stokes_einstein`

The diffusion coefficient of a sphere of hydrodynamic radius r in a fluid of viscosity eta at temperature T.

$$D = \frac{k_B T}{6\pi\eta r}$$

symbol		meaning	SI unit
`T`	in	absolute temperature (degC is converted)	K
`eta`	in	viscosity of the medium	Pa*s
`r`	in	hydrodynamic radius	m
`D`	out	diffusion coefficient	m^2/s
`D_um2_s`	out	diffusion coefficient in um^2/s	—
`friction`	out	friction coefficient 6 pi eta r in kg/s	—

Applies when: T, eta, r > 0; a sphere at low Reynolds number.

```
$ cell-molecular-biology solve cmb_stokes_einstein "T=298.15 K" "eta=0.89 mPa*s" "r=2.5 nm"
D = 9.814924382e-11 m^2/s
D_um2_s = 98.14924382
friction = 4.194026193e-11
  by: Stokes-Einstein diffusion coefficient  (D = k T / (6 pi eta r))
  T = 298.15 K
  eta = 0.00089 Pa*s
  r = 2.5e-09 m
  assumes: T, eta, r > 0; a sphere at low Reynolds number
```

In the worked example, T = 298.15 K, eta = 0.89 mPa*s, r = 2.5 nm go through the equation above and give D = 9.814924382e-11 m^2/s, D_um2_s = 98.14924382, friction = 4.194026193e-11. It applies only when T, eta, r > 0; a sphere at low Reynolds number: outside that it says what is wrong instead of returning a number.

Time to diffuse a distance — `cmb_diffusion_time`

The time for the root-mean-square displacement of a diffusing molecule to reach a distance x in 1, 2 or 3 dimensions.

$$t = \frac{x^2}{2dD}$$

symbol		meaning	SI unit
`x`	in	distance (root-mean-square displacement to reach)	m
`D`	in	diffusion coefficient	m^2/s
`dims`	in	number of dimensions (1, 2 or 3)	—
`t`	out	diffusion time	s
`t_min`	out	diffusion time in minutes	—

Applies when: x > 0, D > 0; dims = 1, 2 or 3.

```
$ cell-molecular-biology solve cmb_diffusion_time "x=10 um" "D=1000 um^2/s" dims=3
t = 0.01666666667 s
t_min = 0.0002777777778
  by: Time to diffuse a distance  (t = x^2 / (2 d D))
  x = 1e-05 m
  D = 1e-09 m^2/s
  assumes: x > 0, D > 0; dims = 1, 2 or 3
```

In the worked example, x = 10 um, D = 1000 um^2/s, dims = 3 go through the equation above and give t = 0.01666666667 s, t_min = 0.0002777777778. It applies only when x > 0, D > 0; dims = 1, 2 or 3: outside that it says what is wrong instead of returning a number.

PCR and gene expression (11)

mRNA and protein after switching a gene on — `cmb_induction_dynamics`

The mRNA and protein numbers a time t after a gene is switched on from zero (constant transcription and translation, first-order decay), and the protein's fraction of its steady state.

$$p(t) = \frac{k_{tl} k_{tx}}{\delta_m} \left[\frac{1 - e^{-\delta_p t}}{\delta_p} - \frac{e^{-\delta_m t} - e^{-\delta_p t}}{\delta_p - \delta_m} \right]$$

symbol		meaning	SI unit
`k_tx`	in	transcription rate	Hz
`delta_m`	in	mRNA decay rate constant	1/s
`k_tl`	in	translation rate per mRNA	Hz
`delta_p`	in	protein decay (plus dilution) rate constant	1/s
`t`	in	time since induction	s
`m`	out	mRNA per cell at time t	—
`p`	out	protein per cell at time t	—
`fraction_of_steady_state`	out	p / p_ss	—

Applies when: all rates > 0; t >= 0; m(0) = p(0) = 0.

```
$ cell-molecular-biology solve cmb_induction_dynamics "k_tx=36 1/h" "delta_m=7.2 1/h" "k_tl=180
1/h" "delta_p=0.36 1/h" "t=2 h"
m = 4.999997213
p = 1219.073084
fraction_of_steady_state = 0.4876292336
  by: mRNA and protein after switching a gene on  (m = (k_tx/delta_m)(1 - e^(-delta_m t)); p =
(k_tl k_tx/delta_m)[(1 - e^(-delta_p t))/delta_p - (e^(-delta_m t) - e^(-delta_p t))/(delta_p -
delta_m)] (t e^(-delta t) when equal))
  k_tx = 0.01 Hz
  delta_m = 0.002 1/s
  k_tl = 0.05 Hz
  delta_p = 0.0001 1/s
  t = 7200 s
  assumes: all rates > 0; t >= 0; m(0) = p(0) = 0
```

In the worked example, k_tx = 36 1/h, delta_m = 7.2 1/h, k_tl = 180 1/h, delta_p = 0.36 1/h, t = 2 h go through the equation above and give m = 4.999997213, p = 1219.073084, fraction_of_steady_state = 0.4876292336. It applies only when all rates > 0; t >= 0; m(0) = p(0) = 0: outside that it says what is wrong instead of returning a number.

mRNA half-life fitted to a decay time course — `cmb_mrna_half_life_fit`

The decay rate constant and half-life of an mRNA (or protein) from levels measured after transcription is stopped, by a least-squares line through ln(level) against time.

$$\ln L = \ln L_0 - kt, \quad t_{1/2} = \frac{\ln 2}{k}$$

symbol		meaning	SI unit
`t`	in	times after the shut-off in minutes	—
`level`	in	measured levels (relative units, positive)	—
`k_per_min`	out	decay rate constant per minute	—
`half_life_min`	out	half-life in minutes	—
`initial_fit`	out	fitted level at t = 0	—
`r2`	out	coefficient of determination of the log-linear fit	—

Applies when: at least 3 points; levels > 0 and decaying; first-order decay.

```
$ cell-molecular-biology solve cmb_mrna_half_life_fit "t=[0, 30, 60, 90, 120]" "level=[100, 70,
50.5, 35.2, 24.8]"
k_per_min = 0.01158700742
half_life_min = 59.82106989
initial_fit = 99.95560576
r2 = 0.9997993322
  by: mRNA half-life fitted to a decay time course  (least squares ln(level) = ln(L0) - k t; t1/2 =
ln 2 / k)
  assumes: at least 3 points; levels > 0 and decaying; first-order decay
```

In the worked example, t = [0, 30, 60, 90, 120], level = [100, 70, 50.5, 35.2, 24.8] go through the equation above and give k_per_min = 0.01158700742, half_life_min = 59.82106989, initial_fit = 99.95560576, r2 = 0.9997993322. It applies only when at least 3 points; levels > 0 and decaying; first-order decay: outside that it says what is wrong instead of returning a number.

PCR amplification after n cycles — `cmb_pcr_amplification`

The number of copies after n PCR cycles from N0 template copies at an amplification efficiency E (1 = perfect doubling).

$$N = N_0(1 + E)^n$$

symbol		meaning	SI unit
`N0`	in	initial template copies	—

symbol		meaning	SI unit
`E`	in	amplification efficiency per cycle (0 to 1, e.g. 95%)	—
`n`	in	number of cycles	—
`N`	out	copies after n cycles	—
`fold`	out	fold amplification (1 + E)^n	—
`log10_fold`	out	log10 of the fold amplification	—

Applies when: N0 > 0; 0 < E <= 1; 0 <= n <= 60; exponential phase (no plateau).

```
$ cell-molecular-biology solve cmb_pcr_amplification N0=100 E=1 n=30
N = 1.073741824e+11
fold = 1073741824
log10_fold = 9.03089987
  by: PCR amplification after n cycles  (N = N0 (1 + E)^n)
  N0 = 100
  E = 1
  assumes: N0 > 0; 0 < E <= 1; 0 <= n <= 60; exponential phase (no plateau)
```

In the worked example, N0 = 100, E = 1, n = 30 go through the equation above and give N = 1.073741824e+11, fold = 1073741824, log10_fold = 9.03089987. It applies only when N0 > 0; 0 < E <= 1; 0 <= n <= 60; exponential phase (no plateau): outside that it says what is wrong instead of returning a number.

PCR cycles needed to reach a number of copies — `cmb_pcr_cycles_needed`

The number of PCR cycles needed to amplify N0 copies to N at efficiency E (exact value and whole cycles, rounded up).

$$n = \frac{\ln(N/N_0)}{\ln(1+E)}$$

symbol		meaning	SI unit
`N0`	in	initial template copies	—
`N`	in	target number of copies	—
`E`	in	amplification efficiency per cycle (0 to 1)	—
`cycles_exact`	out	cycles needed (real value)	—
`cycles`	out	whole cycles needed	—

Applies when: N > N0 > 0; 0 < E <= 1.

```
$ cell-molecular-biology solve cmb_pcr_cycles_needed N0=100 N=1e+10 E=0.95
cycles_exact = 27.58291489
cycles = 28
  by: PCR cycles needed to reach a number of copies  (n = ln(N/N0) / ln(1 + E))
  N0 = 100
  N = 1e+10
  E = 0.95
  assumes: N > N0 > 0; 0 < E <= 1
```

In the worked example, N0 = 100, N = 1e+10, E = 0.95 go through the equation above and give cycles_exact = 27.58291489, cycles = 28. It applies only when N > N0 > 0; 0 < E <= 1: outside that it says what is wrong instead of returning a number.

qPCR relative expression by the delta-delta Ct method — `cmb_qpcr_delta_delta_ct`

The fold change in a target gene's expression between treated and control samples, normalized to a reference gene, by 2^-ddCt (100 % efficiency assumed).

$$\Delta\Delta C_t = \Delta C_{t,treated} - \Delta C_{t,control}, \quad \text{fold} = 2^{-\Delta\Delta C_t}$$

symbol		meaning	SI unit
`Ct_target_treated`	in	Ct of the target gene in the treated sample	—
`Ct_ref_treated`	in	Ct of the reference gene in the treated sample	—
`Ct_target_control`	in	Ct of the target gene in the control sample	—
`Ct_ref_control`	in	Ct of the reference gene in the control sample	—
`dCt_treated`	out	Ct target - Ct reference, treated	—
`dCt_control`	out	Ct target - Ct reference, control	—
`ddCt`	out	dCt treated - dCt control	—
`fold_change`	out	relative expression 2^-ddCt	—
`log2_fold_change`	out	log2 fold change (-ddCt)	—

Applies when: 0 < Ct <= 60; efficiencies of target and reference near 100 %.

```
$ cell-molecular-biology solve cmb_qpcr_delta_delta_ct Ct_target_treated=24.5 Ct_ref_treated=18
Ct_target_control=27 Ct_ref_control=18.2
dCt_treated = 6.5
dCt_control = 8.8
ddCt = -2.3
fold_change = 4.924577653
log2_fold_change = 2.3
  by: qPCR relative expression by the delta-delta Ct method  (ddCt = (Ct_t - Ct_r)_treated - (Ct_t
- Ct_r)_control; fold = 2^-ddCt)
  Ct_target_treated = 24.5
  Ct_ref_treated = 18
  Ct_target_control = 27
  Ct_ref_control = 18.2
  assumes: 0 < Ct <= 60; efficiencies of target and reference near 100 %
```

In the worked example, Ct_target_treated = 24.5, Ct_ref_treated = 18, Ct_target_control = 27, Ct_ref_control = 18.2 go through the equation above and give dCt_treated = 6.5, dCt_control = 8.8, ddCt = -2.3, fold_change = 4.924577653. It applies only when 0 < Ct <= 60; efficiencies of target and reference near 100 %: outside that it says what is wrong instead of returning a number.

qPCR relative expression with efficiency correction (Pfaffl) — `cmb_qpcr_pfaffl_ratio`

The efficiency-corrected expression ratio of a target gene in a sample versus a control, normalized to a reference gene (Pfaffl 2001).

$$\text{ratio} = \frac{(1+E_t)^{\Delta C_{t,target}}}{(1+E_r)^{\Delta C_{t,ref}}}$$

symbol		meaning	SI unit
`E_target`	in	amplification efficiency of the target assay (0 to 1)	—
`E_ref`	in	amplification efficiency of the reference assay (0 to 1)	—
`Ct_target_control`	in	target Ct in the control	—
`Ct_target_sample`	in	target Ct in the sample	—
`Ct_ref_control`	in	reference Ct in the control	—
`Ct_ref_sample`	in	reference Ct in the sample	—
`ratio`	out	expression ratio sample/control	—
`log2_ratio`	out	log2 of the ratio	—
`dCt_target`	out	Ct control - Ct sample for the target	—
`dCt_ref`	out	Ct control - Ct sample for the reference	—

Applies when: 0 < E <= 1; 0 < Ct <= 60.

```
$ cell-molecular-biology solve cmb_qpcr_pfaffl_ratio E_target=0.95 E_ref=0.9 Ct_target_control=26
Ct_target_sample=23.5 Ct_ref_control=18 Ct_ref_sample=18.3
ratio = 6.437431321
log2_ratio = 2.686485135
dCt_target = 2.5
dCt_ref = -0.3
  by: qPCR relative expression with efficiency correction (Pfaffl)  (ratio = (1 +
E_t)^(Ct_t,control - Ct_t,sample) / (1 + E_r)^(Ct_r,control - Ct_r,sample))
  E_target = 0.95
  E_ref = 0.9
  Ct_target_control = 26
  Ct_target_sample = 23.5
  Ct_ref_control = 18
  Ct_ref_sample = 18.3
  assumes: 0 < E <= 1; 0 < Ct <= 60
```

In the worked example, E_target = 0.95, E_ref = 0.9, Ct_target_control = 26, Ct_target_sample = 23.5, Ct_ref_control = 18, Ct_ref_sample = 18.3 go through the equation above and give ratio = 6.437431321, log2_ratio = 2.686485135, dCt_target = 2.5, dCt_ref = -0.3. It applies only when 0 < E <= 1; 0 < Ct <= 60: outside that it says what is wrong instead of returning a number.

qPCR standard curve: slope, intercept and efficiency — `cmb_qpcr_standard_curve`

The least-squares standard curve Ct = m log10(quantity) + b from a dilution series, its r^2 and the amplification efficiency 10^(-1/m) - 1.

$$C_t = m \log_{10} q + b, \quad E = 10^{-1/m} - 1$$

symbol		meaning	SI unit
`quantity`	in	starting quantities of the standards (copies or ng)	—

symbol		meaning	SI unit
`Ct`	in	measured Ct of each standard	—
`slope`	out	slope m (about -3.32 at 100 %)	—
`intercept`	out	intercept b (Ct of one unit)	—
`r2`	out	coefficient of determination	—
`efficiency`	out	efficiency 10^(-1/m) - 1	—
`amplification_factor`	out	amplification per cycle 10^(-1/m)	—

Applies when: at least 3 standards; quantities > 0; negative slope.

```
$ cell-molecular-biology solve cmb_qpcr_standard_curve "quantity=[1000000.0, 100000, 10000, 1000,
100]" "Ct=[15.1, 18.5, 21.8, 25.2, 28.5]"
slope = -3.35
intercept = 35.22
r2 = 0.9999732687
efficiency = 0.988416978
amplification_factor = 1.988416978
  by: qPCR standard curve: slope, intercept and efficiency  (least squares Ct = m log10 q + b; E =
10^(-1/m) - 1)
  assumes: at least 3 standards; quantities > 0; negative slope
```

In the worked example, quantity = [1e+06, 100000, 10000, 1000, 100], Ct = [15.1, 18.5, 21.8, 25.2, 28.5] go through the equation above and give slope = -3.35, intercept = 35.22, r2 = 0.9999732687, efficiency = 0.988416978. It applies only when at least 3 standards; quantities > 0; negative slope: outside that it says what is wrong instead of returning a number.

Quantity from a Ct and a standard curve — `cmb_qpcr_quantify`

The starting quantity of an unknown from its Ct and the standard curve Ct = m log10 q + b.

$$q = 10^{(C_t - b)/m}$$

symbol		meaning	SI unit
`slope`	in	standard-curve slope m (negative)	—
`intercept`	in	standard-curve intercept b	—
`Ct`	in	Ct of the unknown	—
`quantity`	out	starting quantity (unit of the standards)	—
`log10_quantity`	out	log10 of the quantity	—

Applies when: slope < 0; Ct inside the range of the standards.

```
$ cell-molecular-biology solve cmb_qpcr_quantify slope=-3.32 intercept=38 Ct=24.7
quantity = 10139.67643
log10_quantity = 4.006024096
  by: Quantity from a Ct and a standard curve  (q = 10^((Ct - b)/m))
  slope = -3.32
  intercept = 38
  Ct = 24.7
  assumes: slope < 0; Ct inside the range of the standards
```

In the worked example, slope = -3.32, intercept = 38, Ct = 24.7 go through the equation above and give quantity = 10139.67643, log10_quantity = 4.006024096. It applies only when slope < 0; Ct inside the range of the standards: outside that it says what is wrong instead of returning a number.

Steady-state mRNA and protein numbers — `cmb_expression_steady_state`

The steady-state numbers of mRNA (k_tx/delta_m) and protein (k_tl m/delta_p) per cell for constant transcription, translation and first-order decay (dilution included), with the half-lives.

$$m^* = \frac{k_{tx}}{\delta_m}, \quad p^* = \frac{k_{tl}\,k_{tx}}{\delta_m\,\delta_p}$$

symbol		meaning	SI unit
`k_tx`	in	transcription rate (mRNA made per cell per unit time)	Hz
`delta_m`	in	mRNA decay rate constant	1/s
`k_tl`	in	translation rate (proteins per mRNA per unit time)	Hz
`delta_p`	in	protein decay (plus dilution) rate constant	1/s
`m_ss`	out	steady-state mRNA per cell	—
`p_ss`	out	steady-state protein per cell	—
`mRNA_half_life`	out	mRNA half-life	s
`protein_half_life`	out	protein half-life	s
`mRNA_half_life_min`	out	mRNA half-life in minutes	—
`protein_half_life_h`	out	protein half-life in hours	—

Applies when: all rates > 0; constant rates, first-order decay.

```
$ cell-molecular-biology solve cmb_expression_steady_state "k_tx=36 1/h" "delta_m=7.2 1/h"
"k_tl=180 1/h" "delta_p=0.36 1/h"
m_ss = 5
p_ss = 2500
mRNA_half_life = 346.5735903 s
protein_half_life = 6931.471806 s
mRNA_half_life_min = 5.776226505
protein_half_life_h = 1.925408835
  by: Steady-state mRNA and protein numbers  (dm/dt = k_tx - delta_m m, dp/dt = k_tl m - delta_p p;
at steady state m = k_tx/delta_m, p = k_tl k_tx/(delta_m delta_p))
  k_tx = 0.01 Hz
  delta_m = 0.002 1/s
  k_tl = 0.05 Hz
  delta_p = 0.0001 1/s
  assumes: all rates > 0; constant rates, first-order decay
```

In the worked example, k_tx = 36 1/h, delta_m = 7.2 1/h, k_tl = 180 1/h, delta_p = 0.36 1/h go through the equation above and give m_ss = 5, p_ss = 2500, mRNA_half_life = 346.5735903 s, protein_half_life = 6931.471806 s. It applies only when all rates > 0; constant rates, first-order decay: outside that it says what is wrong instead of returning a number.

Time to transcribe a gene — `cmb_transcription_time`

The time for RNA polymerase to transcribe a gene of n nucleotides at an elongation rate, and, with an initiation rate, the number of polymerases on the gene and their spacing.

$$t = \frac{n}{v}, \quad N_{pol} = k_{init}\, t$$

symbol		meaning	SI unit
`n`	in	gene (transcript) length in nucleotides	—
`rate`	in	elongation rate in nucleotides per second	—
`k_init`	in	initiation rate (optional) (optional)	Hz
`t`	out	transcription time	s
`t_min`	out	time in minutes	—
`polymerases`	out	polymerases on the gene at steady state (when k_init is given)	—
`spacing_nt`	out	mean spacing between polymerases in nt (when k_init is given)	—

Applies when: n >= 1; rate > 0; constant elongation rate.

```
$ cell-molecular-biology solve cmb_transcription_time n=20000 rate=40
t = 500 s
t_min = 8.333333333
  by: Time to transcribe a gene  (t = n / rate; polymerases = k_init t; spacing = rate / k_init)
  rate = 40
  assumes: n >= 1; rate > 0; constant elongation rate
```

In the worked example, n = 20000, rate = 40 go through the equation above and give t = 500 s, t_min = 8.333333333. It applies only when n >= 1; rate > 0; constant elongation rate: outside that it says what is wrong instead of returning a number.

Time to translate a protein — `cmb_translation_time`

The time for a ribosome to synthesise a protein of n_aa amino acids at an elongation rate, the coding length with the stop codon, and, with an initiation rate, the ribosomes per mRNA and their spacing.

$$t = \frac{n_{aa}}{v}, \quad N_{rib} = k_{init}\, t$$

symbol		meaning	SI unit
`n_aa`	in	protein length in amino acids	—
`rate`	in	elongation rate in amino acids per second	—
`k_init`	in	initiation rate per mRNA (optional) (optional)	Hz
`t`	out	translation time	s
`t_min`	out	time in minutes	—
`coding_nt`	out	coding sequence length in nt including the stop codon	—
`ribosomes`	out	ribosomes per mRNA at steady state (when k_init is given)	—
`spacing_codons`	out	mean spacing between ribosomes in codons (when k_init is given)	—

Applies when: n_aa >= 1; rate > 0.

```
$ cell-molecular-biology solve cmb_translation_time n_aa=300 rate=6
t = 50 s
t_min = 0.8333333333
coding_nt = 903
  by: Time to translate a protein  (t = n_aa / rate; coding length = 3 (n_aa + 1); ribosomes =
k_init t)
  rate = 6
  assumes: n_aa >= 1; rate > 0
```

In the worked example, n_aa = 300, rate = 6 go through the equation above and give t = 50 s, t_min = 0.8333333333, coding_nt = 903. It applies only when n_aa >= 1; rate > 0: outside that it says what is wrong instead of returning a number.

Solution biochemistry (13)

Buffer capacity of a weak-acid buffer — `cmb_buffer_capacity`

The buffer capacity beta = dCb/dpH (strong base per unit pH change) of a monoprotic buffer of total concentration C at a pH, including water, and the buffer's own contribution (25 degC, Kw = 1e-14).

$$\beta = 2.303\left([\mathrm{H^+}] + \frac{K_w}{[\mathrm{H^+}]} + \frac{C K_a [\mathrm{H^+}]}{(K_a + [\mathrm{H^+}])^2}\right)$$

symbol		meaning	SI unit
`C`	in	total buffer concentration	mol/m^3
`pKa`	in	pKa of the buffer acid	—
`pH`	in	pH	—

symbol		meaning	SI unit
`beta`	out	buffer capacity per pH unit	mol/m^3
`beta_mM`	out	buffer capacity in mM per pH unit	—
`beta_buffer_mM`	out	contribution of the buffer alone in mM per pH unit	—

Applies when: C > 0; monoprotic acid; Kw = 1e-14 (25 degC), ideal solution.

```
$ cell-molecular-biology solve cmb_buffer_capacity "C=100 (mmol/L)" pKa=7.21 pH=7.21
beta = 57.56514274 mol/m^3
beta_mM = 57.56514274
beta_buffer_mM = 57.56462732
  by: Buffer capacity of a weak-acid buffer  (beta = 2.303 ([H+] + Kw/[H+] + C Ka [H+] / (Ka +
[H+])^2))
  C = 100 mol/m^3
  pKa = 7.21
  pH = 7.21
  assumes: C > 0; monoprotic acid; Kw = 1e-14 (25 degC), ideal solution
```

In the worked example, C = 100 (mmol/L), pKa = 7.21, pH = 7.21 go through the equation above and give beta = 57.56514274 mol/m^3, beta_mM = 57.56514274, beta_buffer_mM = 57.56462732. It applies only when C > 0; monoprotic acid; Kw = 1e-14 (25 degC), ideal solution: outside that it says what is wrong instead of returning a number.

Buffer pH after adding strong acid or base — `cmb_buffer_after_addition`

The pH of a buffer (total concentration C, volume V, initial pH0) after adding an amount of strong acid or strong base, by Henderson-Hasselbalch on the new amounts.

$$\mathrm{pH} = \mathrm{p}K_a + \log_{10}\frac{n_{A^-} \pm n}{n_{HA} \mp n}$$

symbol		meaning	SI unit
`C`	in	total buffer concentration	mol/m^3
`V`	in	buffer volume	m^3
`pH0`	in	initial pH	—
`pKa`	in	pKa of the buffer acid	—
`n_add`	in	amount of strong acid or base added	mol
`added`	in	strong acid or strong base	—
`pH`	out	new pH	—
`delta_pH`	out	change in pH	—
`n_base`	out	base form after the addition	mol
`n_acid`	out	acid form after the addition	mol

Applies when: C, V, n_add > 0; the addition smaller than the form it consumes; Henderson-Hasselbalch valid.

```
$ cell-molecular-biology solve cmb_buffer_after_addition "C=100 (mmol/L)" "V=100 mL" pH0=7.21
pKa=7.21 "n_add=1 mmol" added=base
pH = 7.386091259
delta_pH = 0.1760912591
n_base = 0.006 mol
n_acid = 0.004 mol
  by: Buffer pH after adding strong acid or base  (split C V by the initial ratio 10^(pH0 - pKa);
base converts HA to A- (acid the reverse); pH = pKa + log10(n_A/n_HA))
  C = 100 mol/m^3
  V = 0.0001 m^3
  pH0 = 7.21
  pKa = 7.21
  n_add = 0.001 mol
  added = base
  assumes: C, V, n_add > 0; the addition smaller than the form it consumes; Henderson-Hasselbalch
valid
```

In the worked example, C = 100 (mmol/L), V = 100 mL, pH0 = 7.21, pKa = 7.21, n_add = 1 mmol, added = base go through the equation above and give pH = 7.386091259, delta_pH = 0.1760912591, n_base = 0.006 mol, n_acid = 0.004 mol. It applies only when C, V, n_add > 0; the addition smaller than the form it consumes; Henderson-Hasselbalch valid: outside that it says what is wrong instead of returning a number.

Buffer pH by the Henderson-Hasselbalch equation — `cmb_henderson_hasselbalch`

The pH of a buffer from the pKa and the concentrations of the conjugate base and the weak acid.

$$\mathrm{pH} = \mathrm{p}K_a + \log_{10}\frac{[\mathrm{A}^-]}{[\mathrm{HA}]}$$

symbol		meaning	SI unit
`pKa`	in	pKa of the weak acid	—
`A`	in	conjugate base concentration	mol/m^3
`HA`	in	weak acid concentration	mol/m^3
`pH`	out	buffer pH	—
`ratio`	out	[A-]/[HA]	—
`fraction_base`	out	fraction in the base form	—

Applies when: A, HA > 0; buffer concentrations well above [H+] and [OH-].

```
$ cell-molecular-biology solve cmb_henderson_hasselbalch pKa=7.21 "A=60 (mmol/L)" "HA=40 (mmol/L)"
pH = 7.386091259
ratio = 1.5
fraction_base = 0.6
  by: Buffer pH by the Henderson-Hasselbalch equation  (pH = pKa + log10([A-]/[HA]))
  pKa = 7.21
  A = 60 mol/m^3
  HA = 40 mol/m^3
  assumes: A, HA > 0; buffer concentrations well above [H+] and [OH-]
```

In the worked example, pKa = 7.21, A = 60 (mmol/L), HA = 40 (mmol/L) go through the equation above and give pH = 7.386091259, ratio = 1.5, fraction_base = 0.6. It applies only when A, HA > 0; buffer concentrations well above [H+] and [OH-]: outside that it says what is wrong instead of returning a number.

Dilution of a stock solution (C1 V1 = C2 V2) — `cmb_c1v1_dilution`

The volume of stock and of diluent needed to make a volume V2 at concentration C2 from a stock at C1.

$$C_1 V_1 = C_2 V_2$$

symbol		meaning	SI unit
`C1`	in	stock concentration	mol/m^3
`C2`	in	final concentration	mol/m^3
`V2`	in	final volume	m^3
`V1`	out	volume of stock	m^3
`V1_uL`	out	volume of stock in uL	—
`V_diluent`	out	volume of diluent	m^3
`dilution_factor`	out	dilution factor C1/C2	—

Applies when: C1, C2, V2 > 0; C2 <= C1; volumes additive.

```
$ cell-molecular-biology solve cmb_c1v1_dilution "C1=1000 (mmol/L)" "C2=50 (mmol/L)" "V2=10 mL"
V1 = 5e-07 m^3
V1_uL = 500
V_diluent = 9.5e-06 m^3
dilution_factor = 20
  by: Dilution of a stock solution (C1 V1 = C2 V2)  (V1 = C2 V2 / C1; diluent = V2 - V1)
  C1 = 1000 mol/m^3
  C2 = 50 mol/m^3
  V2 = 1e-05 m^3
  assumes: C1, C2, V2 > 0; C2 <= C1; volumes additive
```

In the worked example, C1 = 1000 (mmol/L), C2 = 50 (mmol/L), V2 = 10 mL go through the equation above and give V1 = 5e-07 m^3, V1_uL = 500, V_diluent = 9.5e-06 m^3, dilution_factor = 20. It applies only when C1, C2, V2 > 0; C2 <= C1; volumes additive: outside that it says what is wrong instead of returning a number.

Ionic strength of a solution — `cmb_ionic_strength`

The ionic strength I = (1/2) sum c z^2 of a solution from the concentration and charge of each ion.

$$I = \tfrac{1}{2} \sum_i c_i z_i^2$$

symbol		meaning	SI unit
`c`	in	ion concentrations in mM (one per ion)	—
`z`	in	charge of each ion (e.g. 1, -1, 2)	—
`I`	out	ionic strength	mol/m^3

symbol		meaning	SI unit
`I_mM`	out	ionic strength in mM	—
`I_M`	out	ionic strength in M	—

Applies when: one charge per concentration; charges non-zero whole numbers.

```
$ cell-molecular-biology solve cmb_ionic_strength "c=[150, 150]" "z=[1, -1]"
I = 150 mol/m^3
I_mM = 150
I_M = 0.15
  by: Ionic strength of a solution  (I = 1/2 sum c_i z_i^2)
  assumes: one charge per concentration; charges non-zero whole numbers
```

In the worked example, c = [150, 150], z = [1, -1] go through the equation above and give I = 150 mol/m^3, I_mM = 150, I_M = 0.15. It applies only when one charge per concentration; charges non-zero whole numbers: outside that it says what is wrong instead of returning a number.

Mass of solute for a solution of given molarity — `cmb_mass_for_molarity`

The mass of a compound to weigh out for a volume V of solution at molar concentration C.

$$m = C V M$$

symbol		meaning	SI unit
`C`	in	desired concentration	mol/m^3
`V`	in	volume of solution	m^3
`MW`	in	molar mass of the compound	kg/mol
`m`	out	mass to weigh	kg
`m_g`	out	mass in grams	—
`m_mg`	out	mass in milligrams	—
`n`	out	amount of substance	mol

Applies when: C, V, MW > 0.

```
$ cell-molecular-biology solve cmb_mass_for_molarity "C=150 (mmol/L)" "V=500 mL" "MW=58.44 g/mol"
m = 0.004383 kg
m_g = 4.383
m_mg = 4383
n = 0.075 mol
  by: Mass of solute for a solution of given molarity  (m = C V MW)
  C = 150 mol/m^3
  V = 0.0005 m^3
  MW = 0.05844 kg/mol
  assumes: C, V, MW > 0
```

In the worked example, C = 150 (mmol/L), V = 500 mL, MW = 58.44 g/mol go through the equation above and give m = 0.004383 kg, m_g = 4.383, m_mg = 4383, n = 0.075 mol. It applies only when C, V, MW > 0: outside that it says what is wrong instead of returning a number.

Molar concentration from mass and volume — `cmb_molarity_from_mass`

The molar concentration of a solution made by dissolving a mass of compound to a final volume V.

$$C = \frac{m}{M\,V}$$

symbol		meaning	SI unit
`m`	in	mass of solute	kg
`MW`	in	molar mass	kg/mol
`V`	in	final volume	m^3
`C`	out	molar concentration	mol/m^3
`C_mM`	out	concentration in mM	—
`C_M`	out	concentration in M	—
`n`	out	amount of substance	mol

Applies when: m, MW, V > 0.

```
$ cell-molecular-biology solve cmb_molarity_from_mass "m=2920 mg" "MW=58.44 g/mol" "V=500 mL"
C = 99.93155373 mol/m^3
C_mM = 99.93155373
C_M = 0.09993155373
n = 0.04996577687 mol
  by: Molar concentration from mass and volume  (C = m / (MW V))
  m = 0.00292 kg
  MW = 0.05844 kg/mol
  V = 0.0005 m^3
  assumes: m, MW, V > 0
```

In the worked example, m = 2920 mg, MW = 58.44 g/mol, V = 500 mL go through the equation above and give C = 99.93155373 mol/m^3, C_mM = 99.93155373, C_M = 0.09993155373, n = 0.04996577687 mol. It applies only when m, MW, V > 0: outside that it says what is wrong instead of returning a number.

Molarity of a per cent (w/v) solution — `cmb_percent_wv_to_molarity`

The molar concentration and grams per litre of a solution given in per cent weight per volume (g per 100 mL), e.g. 0.9 % NaCl.

$$C = \frac{10 \times (\%\,w/v)}{M}$$

symbol		meaning	SI unit
`p`	in	concentration in per cent w/v (written with %, e.g. 0.9%)	—
`MW`	in	molar mass of the solute	kg/mol
`C`	out	molar concentration	mol/m^3
`C_mM`	out	concentration in mM	—
`C_M`	out	concentration in M	—

symbol		meaning	SI unit
`mass_concentration`	out	mass concentration	kg/m^3
`g_per_L`	out	grams per litre	—

Applies when: 0 < p < 100 %; MW > 0.

```
$ cell-molecular-biology solve cmb_percent_wv_to_molarity p=0.009 "MW=58.44 g/mol"
C = 154.0041068 mol/m^3
C_mM = 154.0041068
C_M = 0.1540041068
mass_concentration = 9 kg/m^3
g_per_L = 9
  by: Molarity of a per cent (w/v) solution  (g/L = 10 x (% w/v); C = (g/L) / MW)
  p = 0.009
  MW = 0.05844 kg/mol
  assumes: 0 < p < 100 %; MW > 0
```

In the worked example, p = 0.009, MW = 58.44 g/mol go through the equation above and give C = 154.0041068 mol/m^3, C_mM = 154.0041068, C_M = 0.1540041068, mass_concentration = 9 kg/m^3. It applies only when 0 < p < 100 %; MW > 0: outside that it says what is wrong instead of returning a number.

Recipe for a buffer of given pH and concentration — `cmb_buffer_preparation`

The concentrations, amounts and (with molar masses) masses of the weak acid and its conjugate base needed for a volume V of buffer at total concentration C and a target pH.

$$\frac{[\mathrm{A^-}]}{[\mathrm{HA}]} = 10^{\mathrm{pH}-\mathrm{p}K_a}$$

symbol		meaning	SI unit
`C`	in	total buffer concentration	mol/m^3
`V`	in	volume of buffer	m^3
`pH`	in	target pH	—
`pKa`	in	pKa of the buffer acid	—
`MW_acid`	in	molar mass of the acid form (optional) (optional)	kg/mol
`MW_base`	in	molar mass of the base form (optional) (optional)	kg/mol
`C_base`	out	base-form concentration	mol/m^3
`C_acid`	out	acid-form concentration	mol/m^3
`n_base`	out	amount of base form	mol
`n_acid`	out	amount of acid form	mol
`ratio`	out	[A-]/[HA]	—
`m_acid_g`	out	grams of the acid form (when MW_acid is given)	—
`m_base_g`	out	grams of the base form (when MW_base is given)	—

Applies when: C, V > 0; Henderson-Hasselbalch valid (pH within about 1 of pKa).

```
$ cell-molecular-biology solve cmb_buffer_preparation "C=100 (mmol/L)" "V=1000 mL" pH=7.4 pKa=7.21
"MW_acid=137.99 g/mol" "MW_base=141.96 g/mol"
C_base = 60.76610641 mol/m^3
C_acid = 39.23389359 mol/m^3
n_base = 0.06076610641 mol
n_acid = 0.03923389359 mol
ratio = 1.548816619
m_acid_g = 5.413884976
m_base_g = 8.626356466
  by: Recipe for a buffer of given pH and concentration  (r = 10^(pH - pKa); C_base = C r/(1 + r);
C_acid = C - C_base; n = C V; m = n M)
  C = 100 mol/m^3
  V = 0.001 m^3
  pH = 7.4
  pKa = 7.21
  MW_acid = 0.13799 kg/mol
  MW_base = 0.14196 kg/mol
  assumes: C, V > 0; Henderson-Hasselbalch valid (pH within about 1 of pKa)
```

In the worked example, C = 100 (mmol/L), V = 1000 mL, pH = 7.4, pKa = 7.21, MW_acid = 137.99 g/mol, MW_base = 141.96 g/mol go through the equation above and give C_base = 60.76610641 mol/m^3, C_acid = 39.23389359 mol/m^3, n_base = 0.06076610641 mol, n_acid = 0.03923389359 mol. It applies only when C, V > 0; Henderson-Hasselbalch valid (pH within about 1 of pKa): outside that it says what is wrong instead of returning a number.

Relative centrifugal force from rotor speed — `cmb_centrifuge_rcf`

The relative centrifugal force (x g) at radius r for a rotor speed in rpm.

$$\mathrm{RCF} = \frac{\omega^2 r}{g} = 1.118 \times 10^{-5}\, r_{cm}\, N_{rpm}^2$$

symbol		meaning	SI unit
`rpm`	in	rotor speed	rad/s
`r`	in	radius from the rotor axis	m
`RCF`	out	relative centrifugal force in multiples of g	—
`rpm_value`	out	rotor speed in rpm	—

Applies when: rpm, r > 0.

```
$ cell-molecular-biology solve cmb_centrifuge_rcf "rpm=3000 rpm" "r=10 cm"
RCF = 1006.419562
rpm_value = 3000
  by: Relative centrifugal force from rotor speed  (RCF = omega^2 r / g = 1.118e-5 r(cm) rpm^2)
  rpm = 314.1592654 rad/s
  r = 0.1 m
  assumes: rpm, r > 0
```

In the worked example, rpm = 3000 rpm, r = 10 cm go through the equation above and give RCF = 1006.419562, rpm_value = 3000. It applies only when rpm, r > 0: outside that it says what is wrong instead of

returning a number.

Rotor speed for a required centrifugal force — `cmb_centrifuge_rpm`

The rotor speed (rpm) that gives a required relative centrifugal force (x g) at radius r.

$$N_{rpm} = \frac{60}{2\pi}\sqrt{\frac{\mathrm{RCF}\, g}{r}}$$

symbol		meaning	SI unit
`RCF`	in	required relative centrifugal force in multiples of g	—
`r`	in	radius from the rotor axis	m
`omega`	out	angular speed	rad/s
`rpm_value`	out	rotor speed in rpm	—

Applies when: RCF, r > 0.

```
$ cell-molecular-biology solve cmb_centrifuge_rpm RCF=1000 "r=10 cm"
omega = 313.1557121 rad/s
rpm_value = 2990.416772
  by: Rotor speed for a required centrifugal force  (rpm = sqrt(RCF g / r) x 60 / (2 pi))
  RCF = 1000
  r = 0.1 m
  assumes: RCF, r > 0
```

In the worked example, RCF = 1000, r = 10 cm go through the equation above and give omega = 313.1557121 rad/s, rpm_value = 2990.416772. It applies only when RCF, r > 0: outside that it says what is wrong instead of returning a number.

Serial dilution series — `cmb_serial_dilution`

The concentrations after each step of a serial dilution with a constant factor, the final concentration and total dilution, and (with the tube volume) the transfer and diluent volumes per step.

$$C_k = \frac{C_0}{f^k}$$

symbol		meaning	SI unit
`C0`	in	starting concentration	mol/m^3
`f`	in	dilution factor per step (e.g. 10)	—
`n`	in	number of dilution steps	—
`V`	in	final volume in each tube (optional) (optional)	m^3
`C_final`	out	concentration after n steps	mol/m^3
`total_dilution`	out	total dilution factor f^n	—
`concentrations_mM`	out	concentration after each step in mM	—
`transfer`	out	volume carried to the next tube (when V is given)	m^3

symbol		meaning	SI unit
`diluent`	out	diluent per tube (when V is given)	m^3

Applies when: C0 > 0; f > 1; 1 <= n <= 30.

```
$ cell-molecular-biology solve cmb_serial_dilution "C0=10 (mmol/L)" f=10 n=4
C_final = 0.001 mol/m^3
total_dilution = 10000
concentrations_mM = [1.0,0.1,0.01,0.001]
  by: Serial dilution series  (C_k = C0 / f^k; transfer = V/f, diluent = V - V/f)
  C0 = 10 mol/m^3
  f = 10
  assumes: C0 > 0; f > 1; 1 <= n <= 30
```

In the worked example, C0 = 10 (mmol/L), f = 10, n = 4 go through the equation above and give C_final = 0.001 mol/m^3, total_dilution = 10000, concentrations_mM = [1, 0.1, 0.01, 0.001]. It applies only when C0 > 0; f > 1; 1 <= n <= 30: outside that it says what is wrong instead of returning a number.

Time to pellet a particle in a centrifuge (k-factor) — `cmb_sedimentation_time`

The time to sediment a particle of sedimentation coefficient s from r_min to r_max at a rotor speed, and the rotor's k-factor (t in hours = k / s).

$$t = \frac{\ln(r_{max}/r_{min})}{\omega^2 s}, \quad k = \frac{\ln(r_{max}/r_{min})}{\omega^2} \cdot \frac{10^{13}}{3600}$$

symbol		meaning	SI unit
`s`	in	sedimentation coefficient in svedbergs	—
`rpm`	in	rotor speed	rad/s
`r_min`	in	starting (minimum) radius	m
`r_max`	in	radius of the pellet (maximum radius)	m
`t`	out	sedimentation time	s
`t_h`	out	time in hours	—
`k_factor`	out	k-factor (t_h = k / s)	—

Applies when: s, rpm > 0; 0 < r_min < r_max; constant s (no diffusion).

```
$ cell-molecular-biology solve cmb_sedimentation_time s=70 "rpm=40000 rpm" "r_min=5 cm" "r_max=10
cm"
t = 5643.521741 s
t_h = 1.567644928
k_factor = 109.735145
  by: Time to pellet a particle in a centrifuge (k-factor)  (dr/dt = s omega^2 r: t =
ln(r_max/r_min) / (omega^2 s); k = ln(r_max/r_min) / omega^2 x 1e13 / 3600)
  s = 70
  rpm = 4188.790205 rad/s
  r_min = 0.05 m
  r_max = 0.1 m
  assumes: s, rpm > 0; 0 < r_min < r_max; constant s (no diffusion)
```

In the worked example, s = 70, rpm = 40000 rpm, r_min = 5 cm, r_max = 10 cm go through the equation above and give t = 5643.521741 s, t_h = 1.567644928, k_factor = 109.735145. It applies only when s, rpm > 0; 0 < r_min < r_max; constant s (no diffusion): outside that it says what is wrong instead of returning a number.

Questions and reports: support@ai999x.com. AI999X — The Future Is Self-Improving.